



Systems Analysis & Design

INS224

Covenant Centre for Open
and Distance eLearning

ccodel.covenantuniversity.edu.ng

Table of Contents

Module One: Introduction to Information System Development Planning	2
Unit 1 — The System Analyst and Information System Development	3
Unit 2 — System Development Life Cycle	14
Unit 3 — System Development Methodology	27
Module Two: Requirements Modeling	43
Unit 1 — Fact Gathering Techniques	44
Unit 2 — Modelling System Requirement With Use Cases	57
Unit 3 — Process Modeling	73

Module One:

Introduction to Information System Development Planning

Unit 1 — The System Analyst and Information System Development

INTRODUCTION

Organisations rely heavily on information systems to improve efficiency, support decision-making, and gain a competitive edge. At the centre of this process is the system analyst, a professional responsible for understanding business needs and translating them into effective technological solutions. This lesson introduces the role of the system analyst, the skills required to perform the job, and the stages involved in developing information systems. By the end, learners will understand how system analysts bridge the gap between users, business processes, and technical teams to ensure that information systems are well-designed, reliable, and aligned with organisational goals.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- explain the key roles of a systems analyst in business
- Describe the analyst's role in a system development project
- State the importance of technical skills, people skills, and business skills for an analyst
- Identify the various types of systems and technologies an analyst might use

1.1.1 What is System Analysis?

System Analysis is the systematic process of studying an existing or proposed system to understand its structure, functions, processes, and interactions in order to identify problems, determine requirements, and recommend improvements or solutions.

In the context of information systems development, system analysis examines how an organisation collects, processes, stores, and uses information to ensure the system effectively supports business objectives.

System Analysis can also be defined as the process of understanding and specifying in detail what an information system should accomplish.

System analysis is all about:

- Understanding the business and its goals and strategies,
- Defining requirements for information systems that support those goals and strategies and
- Supporting the business.

Purpose of System Analysis

- To ensure that the system meets user and organisational needs
- To reduce the risk of system failure
- To provide a clear foundation for system design and development
- To improve efficiency, accuracy, and decision-making within the organization

System Design is the phase of system development that involves translating the requirements identified during system analysis into a detailed blueprint for building the system. It defines how the system will be structured, how its components will interact, and how it will operate to meet specified functional and non-functional requirements.

Systems Design specifies in detail how the many components of an information system should be physically implemented.

1.1.2 Who is a Systems Analyst?

A Systems Analyst is a professional who investigates, analyses, and evaluates an organisation's information systems and business processes with the objective of designing or improving systems that effectively support organisational goals.

The systems analyst acts as a link between business users and technical development teams, ensuring that business needs are accurately translated into technical specifications.

Roles and Responsibilities of a Systems Analyst

1. Requirements Analysis: The analyst gathers, analyses, and documents user and system requirements using structured techniques.
2. System Evaluation: Analyst studies existing systems to identify problems, inefficiencies, and areas for improvement.
3. Feasibility Studies: Analyst assesses technical, economic, operational, legal, and schedule feasibility of proposed systems.
4. System Modelling and Documentation: The analyst prepares models such as Data Flow Diagrams (DFDs), Use Case Diagrams, and Entity-Relationship Diagrams (ERDs).
5. Liaison Role: The analyst handles communication between system stakeholders to ensure mutual understanding.
6. Solution Recommendation: Proposes suitable system solutions, including hardware, software, and process improvements.
7. Support During Development and Implementation: Assists in system design, testing, user training, and system deployment.

We can define a systems analyst as a business professional who uses analysis and design techniques to solve business problems using information technology. Essentially, a system analyst is a **problem solver**.

An analyst's role requires extensive technical, business, and people knowledge and skills to solve business problems.

How does an analyst solve problems?

The analyst must first understand the problem and learn everything possible about it, including:

- Who is involved,
- What business processes come into play, and
- What other systems would be affected by solving the problem?

Then the analyst needs to confirm with management that the benefits of solving the problem outweigh the costs.

If solving the problem is feasible, the analyst defines in detail what is required to solve it:

- What specific objectives must be satisfied,
- What data needs to be stored and used,
- What processing must be done to the data, and what outputs must be produced.

What needs to be done must be defined first. How it will be done is not important yet.

Many different alternatives must be considered, and the challenge is to select the best, that is, the solution with the fewest risks and most benefits.

After the systems analyst has determined the best solution in consultation with management, the design details must then be worked out.

The analyst is now concerned with creating a blueprint (design specifications) for the new system.

Systems design specifications describe the construction details of all system components, including databases, networks, program modules, operating procedures, user interfaces, and conversion plans.

After detailed design comes programming. Building a system based on incomplete or misunderstood requirements may lead to

- Over budgeting,
- Late delivery,
- Or delivery of a system that doesn't fully solve the problems it was intended to address.

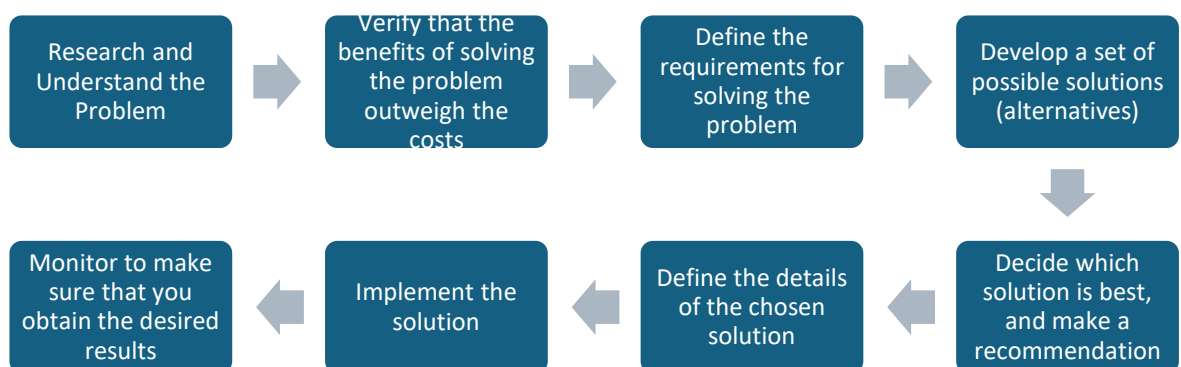


Figure 1: Analyst Approach to Solving Problems

In-Text Questions (ITQs)

1. What is the role of a Systems Analyst in information system development
2. List the steps the analyst follows in solving the problem

In-Text Answers (ITAs)

1. The analyst acts as a bridge between users and technical teams. The analyst studies existing systems and business processes to identify problems and gather user requirements. These requirements are carefully analysed and documented to ensure clarity and accuracy. The systems analyst also conducts feasibility studies to determine whether proposed solutions are practical and cost-effective. By preparing system models and specifications, the analyst guides system design and development, ensuring that the final system meets user needs and supports organisational objectives.
2. Research and Understand the Problem
 - Verify that the benefits of solving the problem outweigh the costs
 - Define the requirements for solving the problem
 - Develop a set of possible solutions (alternatives)
 - Decide which solution is best, and make a recommendation
 - Define the details of the chosen solution
 - Implement the solution
 - Monitor to make sure that you obtain the desired results

1.1.3 Information Systems –the Business Solution

A systems analyst is described as a business problem solver, and the solution to the problem is usually an **information system**.

Some basic information system concepts are:

A system is a collection of interrelated components that function together to achieve some outcome.

An information system is a collection of interrelated components that collect, process, store, and provide as output the information needed to complete business tasks. It can also be described as an arrangement of people, data, processes, and information technology that interact to collect, process, store, and provide as output the information needed to support an organisation.

Its purpose is to process input, maintain data, produce reports, handle queries, handle online transactions, and generate other outputs.

Any system can have subsystems. A subsystem is a system that is part of another system, so subsystems might be one way to think of a system's components.

For example, a sales management system might be one subsystem of a customer relationship management (CRM) system.

Another subsystem in the CRM might enable customers to view past and current orders, track order fulfilment and shipping, and modify their account information.

The view of a system as a collection of subsystems is very useful to the analyst.

It enables the analyst to focus attention on a single area of a business or organization, a group of related areas, or the interfaces among areas.

This approach to dividing a system into components is called functional decomposition.

Figure 2 shows how one system can be divided, or decomposed, into subsystems, which in turn can be further decomposed.

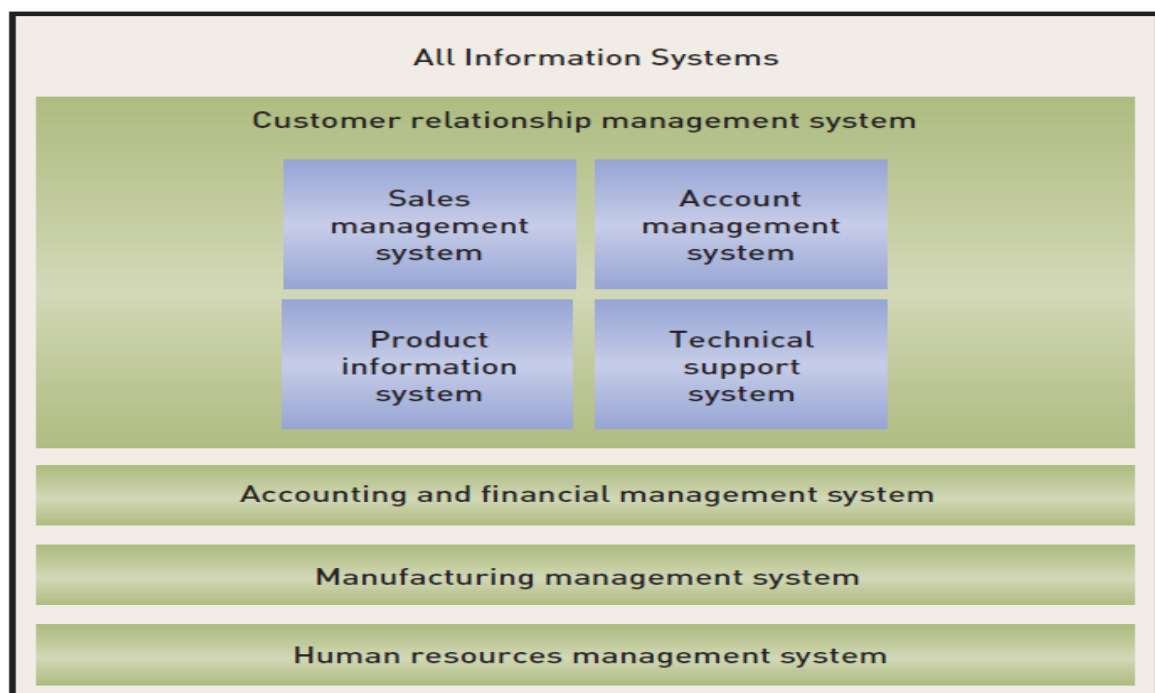


Figure 2: Information systems and subsystems.

1.1.4 Required skills of systems analysts

A **Systems Analyst** requires a combination of technical, analytical, and interpersonal skills to effectively analyse, design, and improve information systems.

They need to be able to understand.

- How to build information systems, which requires quite a bit of technical knowledge.
- The business they are working for and how the business uses each of the types of systems.
- Quite a bit about people and the way they work.

People are the source of requirements information, the labour that builds systems, and the ultimate users of the information system.

Technical knowledge and skills.

A systems analyst needs technical expertise, including knowledge of information systems, software development processes, databases, operating systems, and networks, to understand and communicate technical requirements.

Even if an analyst is not involved in activities such as programming, network design, or hardware configuration, it is still crucial to have an understanding of different types of technology - what they are used for, how they work, and how they are evolving.

A systems analyst should understand the fundamentals of the following:

- Computers and how they work
- File, database, and storage hardware and software
- Input and output hardware and software
- Computer networks and protocols
- Programming languages, operating systems, and utilities
- Communication and collaboration technology such as digital telephones, videoconferencing, and Web-based document management systems.

A systems analyst also needs to know a lot about **tools** and **techniques** for developing systems.

- **Tools:** are software products that are used to develop analysis and design specifications and completed system components.
- **Techniques:** are strategies for completing specific system development activities.

Some tools used in system development include the following:

- **Software packages** such as Microsoft Access and Adobe Dreamweaver that can be used to implement small systems or develop subsystems

- **Integrated development environments (IDEs)** such as Oracle JDeveloper and Microsoft Visual Studio that support program development, database design, software testing, and system deployment.
- **Computer-aided visual modelling tools** such as Rational XDE Modeller, Visible Analyst, and Embarcadero Describe, that help analysts create, store, modify, and manage system specifications and sometimes generate programs, databases, Web-based interfaces, and other software components.

Techniques are strategies for completing specific system development activities.

- How do you plan and manage a system development project?
- How do you define requirements?
- How do you design user interactions using design principles and best practices?
- How do you complete implementation and testing?
- How do you install and support a new information system?

Some examples of techniques include the following:

- Project planning techniques
- Cost/benefit analysis techniques
- Interviewing techniques
- Requirements modelling techniques
- Architectural design techniques
- Network configuration techniques
- Database design techniques

Business knowledge and skills

Understanding of business processes, organisational structures, and industry practices to align systems with organisational goals. The following are examples:

- What business functions do organisations perform?
- How are organisations structured?
- How are organisations managed?
- What type of work goes on in organisations (finance, manufacturing, marketing, customer service, and so on).

Some specifics the analyst needs to know about the company include the following:

- What does the specific organisation do?
- What makes it successful?

- What are its strategies and plans?
- What are its traditions and values?

People's knowledge and skills

Interpersonal skills are the analyst's most important skills, because analysts rely on others, including managers, users, programmers, technical specialists, customers, and vendors, to take a system from initial idea to final implementation. The analyst, therefore, needs the ability to work collaboratively with diverse stakeholders, manage expectations, and resolve conflicts.

The analyst is a translator for all project participants, translating

- Business objectives into functional requirements,
- User needs to system specifications, and
- Technical jargon and details into terms that nontechnical personnel can easily understand.

The analyst must be an effective communicator in many contexts, including conversations, interviews, technical reviews, and formal presentations.

In-Text Questions (ITQs)

1. What are the roles and responsibilities of a Systems Analyst in an organisation?
2. Why are communication skills important for a systems analyst?

In-Text Answers (ITAs)

1. The roles and responsibilities of a Systems Analyst include studying existing systems to identify problems, gathering and analysing user requirements, and conducting feasibility studies for proposed solutions. The analyst documents system specifications, prepares system models, and serves as a link between users and technical development teams. The systems analyst also supports system design, testing, implementation, and user training to ensure that the developed system meets organizational objectives and user needs.
2. Communication skills are important because a systems analyst must interact with users, managers, and technical staff to gather requirements, explain system concepts, and document specifications clearly. Effective

communication helps prevent misunderstandings and ensures that system requirements accurately reflect user needs.

1.1.5 Conclusion

In this unit, you have been introduced to the concepts of system analysis and design, which are fundamental to the development of information systems that support organisational objectives. The systems analyst plays a central role in ensuring that business problems are correctly understood, feasible solutions are identified, and system requirements are accurately translated into practical designs. A successful analyst must possess a balanced combination of technical expertise, business understanding, and strong interpersonal skills to manage complexity, facilitate communication, and guide systems from concept to implementation. When system analysis is carried out thoroughly and systematically, it significantly reduces project risks, improves system quality, and increases the likelihood that the final system will deliver real value to the end users.

Summary

In this unit, you have learnt the role of the systems analyst in information systems development. System analysis is a structured process for understanding business needs, identifying problems, defining requirements, and recommending effective solutions that align information systems with organisational goals. System analysis is followed by system design, which translates the analysed requirements into detailed implementation blueprints. You also learned about the systems analyst, a business problem solver who bridges the gap between users and technical teams. The analyst's roles, responsibilities, and problem-solving approach are outlined. The unit discusses information systems as business solutions composed of interrelated subsystems and emphasises functional decomposition as a key analytical technique. Finally, this unit highlights the technical, business, and people skills required of systems analysts, noting the importance of tools and techniques in delivering successful information systems.

Self-Assessment Questions

1. What are some of the things an analyst needs to understand about people?
2. Define information system.

3. List the six fundamental technologies an analyst needs to understand.
4. List four types of tools the analyst needs to use to develop systems.

Unit Activities

1. Read the content of Unit 1 in the study guide. Consult the recommended texts and research further online. (90 minutes)
2. Complete the self-introductory forum. Original post by Thursday. (10 minutes). Complete replies to 2 of your classmates by Sunday at 11:59 p.m. (10 minutes)
3. TUTOR MARKED ASSIGNMENT: Complete the following and submit for grading. (60 minutes)

Tutor-Marked Assignment

1. Give an example of a business problem.
2. What are the main steps followed when solving a problem?
3. What are some of the things an analyst needs to understand about businesses and organisations in general?
4. What types of information systems are found in most organisations?

References/Further Readings

- System Analysis and Design, Alan Dennis, Barbara H. Wixom, and Robert M. Roth, John Wiley & Sons, 5th Edition, 2012.
- System Analysis and Design Methods, Jeffery L. Whitten and Lonnie D. Bentley, 7th Edition, McGraw-Hill- Irwin, 2007
- Object-Oriented Systems Analysis and Design Using UML, Simon Bennett, Steve McRob and Ray farmer, McGraw-Hill, Second Edition, 2002.

Unit 2 — System Development Life Cycle

INTRODUCTION

The System Development Life Cycle (SDLC) is a framework for planning the development of an information system in an organised manner. It provides a step-by-step approach that guides system developers and analysts from the identification of a business need through the delivery and support of a completed system. Information systems are important to efficient operations, decision-making, and competitive advantage. However, developing such systems without a clear methodology often leads to cost overruns, delayed delivery, or systems that fail to meet user requirements. The SDLC addresses these challenges by defining distinct phases, activities, and deliverables that ensure quality and control throughout the system development process. This unit introduces the SDLC, explains its key phases, and highlights the activities and deliverables at each phase.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Discuss the principles of successful system development
- Describe the two overall approaches used to develop information systems: the predictive and the adaptive approach
- Explain when to use an adaptive approach to the SDLC in place of the more predictive traditional SDLC
- Explain various phases of the development life cycle (SDLC)

1.2.1 System Development

One of the fundamental concepts in information system development is the **systems development life cycle**. Businesses and organisations use information systems to support the processes they need to carry out their functions.

The entire process of building, deploying, using, and updating an information system is called the **systems development life cycle, or SDLC**.

1.2.2 Principles of a Successful Systems Development

- **Get the user involved:** Involving users throughout the processes helps ensure that requirements are accurate, reduces resistance to change, and increases user acceptance of the final system.
- **Use a problem-solving approach:** System development should focus on solving real business problems rather than merely applying technology. Developers and analysts must first understand the problem, evaluate alternative solutions, and then select the option that provides the greatest benefit with the least risk.
- **Establish phases and activities:** Dividing system development into clearly defined phases and activities provides structure and control. This phased approach makes projects easier to manage, monitor, and evaluate, ensuring that each step is completed before moving to the next.
- **Establish standards for development and documentation:** Standards ensure consistency, quality, and clarity across system development activities. Using standard methods, tools, and documentation formats makes systems easier to understand, maintain, and enhance over time.
- **Justify systems as capital investments:** Information systems require significant financial resources and should be evaluated like other capital investments. Cost-benefit analysis helps management determine whether the expected benefits justify the costs and risks involved.
- **Don't be afraid to cancel or revise scope:** If a project becomes too costly, risky, or no longer aligns with organisational goals, it may be necessary to revise its scope or cancel it entirely. Early termination or scope adjustment can prevent further resource waste.
- **Divide and conquer:** Breaking a large system into smaller, manageable components or subsystems simplifies development and reduces complexity. This principle allows teams to focus on individual parts while ensuring the overall system works as a whole.

- **Design systems for growth and change:** Systems should be flexible and scalable to accommodate future business expansion, technological advancements, and changing user requirements. Designing for change extends the system's useful life and protects the organisation's investment.
- **Proper planning and project management:** Effective planning and project management ensure that system development stays on schedule, within budget, and meets quality expectations. This includes setting realistic timelines, allocating resources, managing risks, and monitoring progress throughout the project.

1.2.3 Approaches to SDLC

In system development, approaches are often grouped into **predictive (traditional)** and **adaptive** models based on how they handle requirements, change, and uncertainty.

A **predictive approach** assumes that the development process can be planned and organised in advance and that the new information system can be developed according to the plan. In this approach, the project progresses through a fixed sequence of phases, with each phase completed before the next begins. Predictive SDLCs are useful for building systems that are well-understood and well-defined.

An adaptive approach is an SDLC approach that is more flexible, assuming the project cannot be planned in full in advance and must be modified as it progresses. The **adaptive approach** is designed to handle **changing requirements** and **uncertainty**. Instead of following a rigid sequence, development is carried out in short cycles or iterations, with continuous user feedback and incremental delivery of system functionality.

In predictive approaches, the first group of activities, usually called project planning, involves planning, organising, and scheduling the project. These activities map out the overall structure of the project.

The next group of activities focuses on understanding the business problem that needs to be solved and defining the business requirements. This set of activities is referred to as **analysis activities**. The intent is to understand exactly what the system must do to support the business processes.

A third group of activities focuses on designing the new system. Those activities, called **design activities**, use the requirements that were defined earlier to develop the program structure and algorithms for the new system.

The fourth group of activities is necessary to build the system. This is called **implementation activities**, and it includes programming, testing, and installing the system for business users.

These four groups of activities (**planning, analysis, design, and implementation**) are referred to as **phases**, and they are the elements that provide the framework for managing a project.

Another phase, called the **support phase**, includes activities to upgrade and maintain the system after deployment.

The support phase is part of the overall SDLC but is not normally considered part of the initial development project.

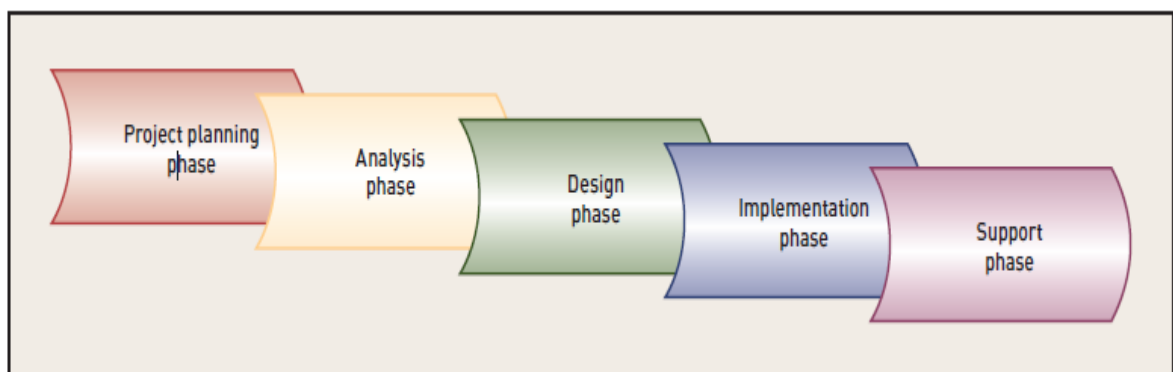


Figure 1: Information Systems Development Phases

In the adaptive SDLC approach, system development activities are not performed once in a fixed sequence. Instead, they are repeated in short cycles known as **iterations** or **sprints**, with each cycle producing a working version of the system or a system component. Typical phases in the adaptive approach can be described as follows:

Initiation and requirements planning: At the start of the project, a high-level system scope is defined. This includes identifying the business problem, the overall objectives, the key stakeholders, and the major system features. Detailed requirements are not fully specified at this stage.

Requirements are captured at a high level and prioritised based on business value and urgency. These requirements are often documented as user stories or feature lists.

Iterative Analysis and Design: Analysis and design are carried out incrementally during each iteration. Only the requirements selected for the current iteration are analysed and designed in detail.

Iterative Development (Construction): System components are developed in short cycles. Each iteration results in a working and testable piece of the system.

Continuous Testing and Quality Assurance: Testing is performed throughout the development process rather than at the end. This ensures early detection and correction of defects.

An example of the adaptive approach is the Spiral model depicted in Figure 2.

Table 1 compares the features of the predictive and adaptive approaches.

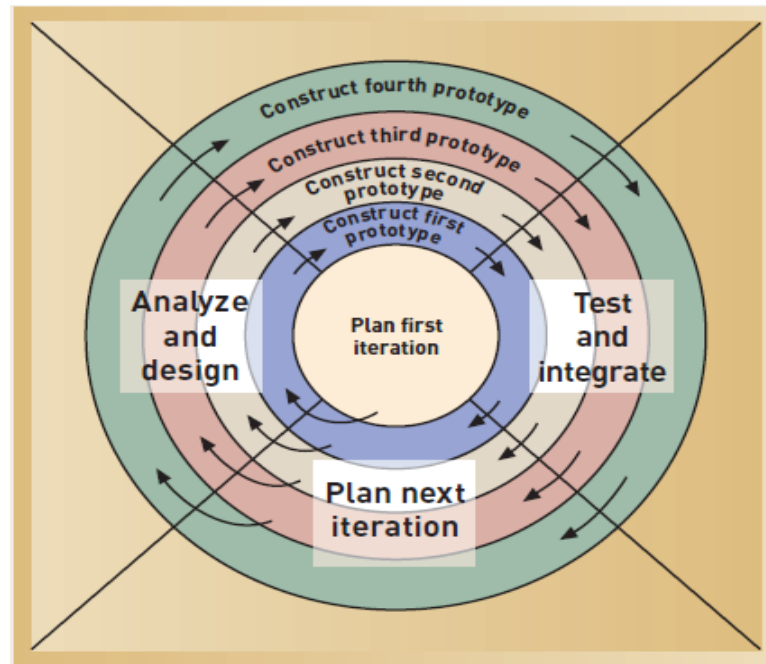


Figure 2: Spiral Life Cycle Model

Table 1: Predictive and Adaptive Approach Compared

Predictive approach	Adaptive approach
Requirements are defined upfront	Requirements evolve over time
Development follows a linear and structured sequence	Development is iterative and incremental
Emphasis on detailed documentation	Strong user involvement and collaboration
Limited flexibility to accommodate changes	Emphasis on working systems over extensive documentation

In-Text Questions (ITQs)

Why is it important to choose an appropriate software development approach for successful system development?

In-Text Answers (ITAs)

The selected approach determines how requirements are gathered, how changes are managed, how risks are controlled, and how the final system is delivered. An approach that aligns with project characteristics such as size, complexity, and user involvement increases the likelihood of meeting project objectives.

For example, a predictive approach is effective when requirements are well understood and unlikely to change. It provides clear structure, documentation, and control for the development process. On the other hand, an adaptive approach is better suited to dynamic business environments that require flexibility and rapid response to change. Selecting the wrong approach can lead to cost overruns, delays, user dissatisfaction, and system failure. Therefore, careful evaluation of organisational needs, project constraints, and stakeholder expectations is important when selecting a development approach to ensure successful system delivery.

1.2.4 Software Development Life Cycle

The entire process of building, deploying, using, and updating an information system is called the systems development life cycle, or SDLC. It ensures that systems are developed in a disciplined and systematic manner to meet user requirements, organisational objectives, and quality standards. The SDLC is commonly organised into five major phases: **Planning, Analysis, Design, Implementation, and Support**.

Planning Phase

The planning phase focuses on identifying the need for a new information system and determining whether the project is feasible and worthwhile. The primary objectives of **project planning** are to identify the scope of the new system, ensure that the project is feasible, and develop a schedule, resource plan, and budget for the remainder of the project.

We identify five activities in project planning:

- **Define the problem** (business problem & scope)
- **Produce the project schedule.**
- **Confirm project feasibility:** Feasibility analysis investigates economic, organisational, technical, resource, and schedule feasibility.
- **Staff the project:** allocating funds, assigning project members, and obtaining other necessary resources such as office and development tools.
- **Launch the project.**

Analysis Phase

The analysis phase involves studying the existing system and business processes to determine detailed system requirements. The primary objective of the analysis activities is to understand and document the business needs and the processing requirements of the new system. Analysis is essentially a discovery process. The key words that drive the activities during analysis are discovery and understanding.

Six primary activities are considered part of this phase:

- **Gather information:** observing the users as they do their work; by interviewing and asking questions of the users; by reading existing documents about procedures, business rules, and job responsibilities; and by reviewing existing automated systems.
- **Define system requirements:** Analysts must review, analyse, and structure the information obtained to develop an overall understanding of the new system's requirements.
- **Prioritise requirements.**
- **Build prototype model for discovery of requirements:** Process modelling, data modelling, etc.
- **Generate and evaluate alternatives.**
- **Review recommendations with management.**

Design Phase

The design phase translates system requirements into detailed specifications that describe how the system will be built. The objective of the design activities is to **design the solution system** based on the requirements defined and decisions made during analysis.

High-level design consists of developing an architectural structure for the software components, databases, user interface, and operating environment.

Low-level design entails developing the detailed algorithms and data structures that are required for software development.

Major activities that must be completed during the design phase:

- Design the application architecture.
- Design and integrate the network.
- Design the user and system interfaces.
- Design and integrate the database.
- Design and integrate the system controls.

Implementation Phase

The implementation phase involves building, testing, and deploying the system based on the design specifications. **Implementation activities** result in the final system being built, tested, and installed. The objective is to **produce a reliable, fully functional information system** and to ensure that all users are **trained** to use the system effectively and efficiently.

Five major activities make up the implementation phase:

- Construct software components.
- Verify and test.
- Convert data.
- Train users and document the system.
- Install the system.

System construction is the first step in the implementation phase. The system is built and tested to ensure it performs as designed.

Since the cost of bugs can be immense, testing is one of the most critical steps in implementation.

Most organisations spend more time and attention on testing than on writing the programs in the first place.

Installation is the process of turning off the old system and turning on the new one.

- It may include a **direct cutover approach** (in which the new system immediately replaces the old system),
- **A parallel conversion approach** (in which both the old and new systems are operated for a month or two until it is clear that there are no bugs in the new system), or

- A **phased conversion strategy** (in which small parts of the new system gradually replace small parts of the old system),
- **Pilot conversion approach** (in which a small group of users within an organisation uses a new system prior to wider use).

One of the most important aspects of conversion is developing a training plan to teach users how to use the new system and help manage the changes it introduces.

Support (Maintenance) Phase

The support phase ensures that the system continues to operate effectively after deployment and adapts to changing needs.

The objective of the support activities is to **keep the system running productively** during the years following its initial installation.

The expectation for most business systems is that the system will last for years. During support, upgrades or enhancements may be carried out to expand the system's capabilities, and they will require their own development projects.

Three major activities occur during support:

- **Maintain the system:** Key tasks in maintaining the system include both fixing the errors (also known as fixing bugs) and making minor adjustments to processing requirements.
- **Enhance the system:** At times, government regulations require new data to be maintained or information to be provided. Also, changes in the business environment, new market opportunities, new competition, or new system infrastructure necessitate major changes to the system.
- **Support the users** (help desk).
 - **Help desk** -the availability of support staff to assist users with any technical or processing problem associated with an information system.
 - Training new users and maintaining current documentation are important elements of this activity.

Table 2: SDLC Phases and Deliverables

Phase	Deliverable
Planning Phase	• Project proposal and recommendation • Investigation Report • Analysis of alternatives • Cost and benefit analysis • Schedule

	• Recommendation
Analysis Phase	• Detailed system requirements • Revised cost and benefits analysis • Revised schedule
Design	• Logical design • Technical design • Test plans • Revised cost and benefits analysis • Revised schedule
Implementation	• System elements • Implementation plans • Contingency plans • End-user training materials • Maintenance plan • Operations plans • Cutover / Phase in
Support	• Maintenance reports • Updated system documentation

In-Text Questions (ITQs)

1. What is the role of feasibility studies in the SDLC?
2. Why is a feasibility study important for system development?

In-Text Answers (ITAs)

1. Feasibility studies are important in the SDLC, particularly during the planning phase. They help to determine whether a proposed system to solve the business problem can be achieved and is worth pursuing. A feasibility study evaluates a project from several perspectives, including technical feasibility (availability of technology and skills), economic feasibility (cost-benefit analysis), operational feasibility (user acceptance and organisational impact), legal feasibility (compliance with laws and regulations), and schedule feasibility (ability to complete the project within the required timeframe).

2. The importance of feasibility studies lies in their ability to reduce risk and prevent wasted resources. By identifying potential constraints and challenges early, management can make informed decisions about whether to proceed, modify, or abandon a project. Feasibility studies ensure that only projects with a reasonable chance of success move forward, thereby increasing the likelihood of delivering effective and sustainable information systems.

1.2.5 Conclusion

The SDLC provides a systematic approach to information system development by dividing the process into manageable phases. Each phase builds on the results of the previous one, ensuring clarity, quality, and control throughout the system's life. Proper application of the SDLC helps organisations develop reliable, efficient, and adaptable information systems that effectively support business operations and decision-making. Together, the principles of successful system development, appropriate development approaches, and the SDLC form the foundation for delivering high-quality, effective, and sustainable information systems.

Summary

In this unit, you have learned about system development processes. Successful system development depends on applying sound principles, selecting an appropriate development approach, and using the System Development Life Cycle (SDLC) in a disciplined manner. The principles of successful system development emphasise strong user involvement, a problem-solving focus, clear phases and standards, justification of systems as capital investments, proper planning and project management, and the need to design systems that are flexible and scalable. They also highlight the importance of managing scope, dividing complex systems into manageable components, and making informed decisions, including cancelling or revising projects when necessary.

System development approaches can be broadly classified into **predictive** (traditional) and **adaptive** approaches. The predictive approach follows a structured, sequential process with clearly defined requirements and extensive documentation, making it suitable for projects with stable and well-understood requirements. In contrast, the adaptive approach is iterative and flexible, allowing requirements to evolve through continuous user feedback and incremental system delivery. Each approach has strengths and limitations, and the choice depends on project characteristics, organisational environment, and the level of uncertainty involved. The SDLC

provides a formal framework for organising system development activities into five key phases: planning, analysis, design, implementation, and support. By following these phases, organisations can manage complexity, reduce risks, and ensure that information systems are aligned with business objectives.

Self-Assessment Questions

1. Compare and contrast the predictive (traditional) SDLC and the adaptive SDLC approaches.
2. Highlight the principles of successful software development
3. Explain the activities in the design phase of SDLC

Unit Activities

1. Read the content of Unit 1 in the study guide. Consult the recommended texts and research further online. (90 minutes)
2. Complete the self-introductory forum. Original post by Thursday. (10 minutes). Complete replies to 2 of your classmates by Sunday at 11:59 p.m. (10 minutes)
3. TUTOR MARKED ASSIGNMENT: Complete the following and submit for grading. (60 minutes)

Tutor-Marked Assignment

1. Explain the Planning and Analysis phases of the System Development Life Cycle (SDLC), highlighting their objectives and key activities.
2. A medium-sized retail company wants to develop a new online sales and inventory management system. The company expects frequent changes in customer requirements, plans to introduce new features regularly, and seeks continuous user feedback throughout development. As a systems analyst,
 - a) Identify the most suitable software development approach for this project.
 - b) Justify your choice by explaining how the selected approach addresses the company's needs.

- c) Outline the key activities that would be carried out using this approach during system development.

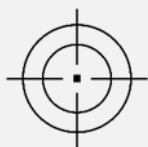
References/Further Readings

- . System Analysis and Design, Alan Dennis, Barbara H. Wixom, and Robert M. Roth, John Wiley & Sons, 5th Edition, 2012.
- System Analysis and Design Methods, Jeffery L. Whitten and Lonnie D. Bentley, 7th Edition, McGraw-Hill- Irwin, 2007
- Object-Oriented Systems Analysis and Design Using UML, Simon Bennett, Steve McRob and Ray farmer, McGraw-Hill, Second Edition, 2002.

Unit 3 — System Development Methodology

INTRODUCTION

In this unit, you will learn the concept of system development methodology and why it is essential for successful system development. This unit explains how methodologies provide structure, consistency, and control for implementing the phases in the system development life cycle. The unit sets the foundation for understanding different software development methodologies and approaches, such as predictive and adaptive models, and how organisations select appropriate methods based on project characteristics.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Describe various approaches (methodologies) that can be used to structure a development project.
- Explain the most suitable scenario to apply each of the methodologies.
- Select the appropriate methodology based on project characteristics

1.3.1 What is Software System Development Methodology?

System development methodology refers to the structured approaches, models, and techniques used to plan, analyse, design, implement, and maintain information systems. Developing information systems without a clear methodology often leads to cost overruns, missed deadlines, and systems that fail to meet user and business requirements.

Systems Development Methodology is the standard process followed by an organisation to conduct all the steps necessary to analyse, design, implement, and maintain information systems. It is a framework used to structure, plan, and control the development of an information system.

Some methodologies include: Waterfall, Rapid Application Development (RAD), Prototyping, Incremental model, Agile, etc.

A. The Waterfall Model

The Waterfall methodology is a traditional, linear approach to system development in which each phase must be completed before the next phase begins. Here, the development proceeds sequentially from one phase to another in a manner like the natural waterfall.

The key deliverables for each phase are usually presented to the approval committee and project sponsor for approval before the project moves to the next phase. As the project progresses from phase to phase, it follows the same waterfall model. Strict monitoring is ensured throughout the project through extensive written documentation, as well as formal reviews and user and information technology management sign-off at the end of each phase.

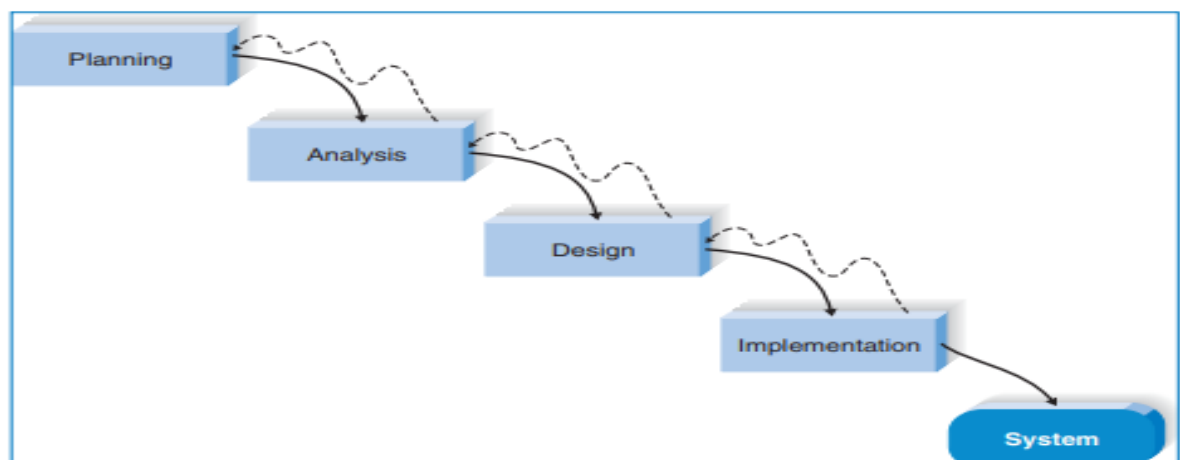


Figure 1: Schematic view of the waterfall model

This model has two very important variants: Parallel Development and V-Model

Advantages of the Waterfall Model

1. Easy to understand, easy to use
2. Provides structure to inexperienced staff
3. The sequential approach and strict controls help to ensure adequate documentation and design reviews, which result in quality, reliability, and maintainability of the developed software
4. Progress of system development is measurable

Disadvantages of the Waterfall Model

1. Inflexible, slow, costly and cumbersome due to significant structure and tight controls
2. Project progresses forward, with only slight movement backward can give a false impression of progress.
3. Little opportunity for the customer to preview the system (until it may be too late).
4. Problems are often not discovered until system testing
5. Difficult to respond to changes.
6. Produces excessive documentation, and keeping it updated as the project progresses is time-consuming

When to Use Waterfall

- Project is for the development of a transaction-oriented batch system.
- Project is large, expensive, and complicated.
- Project has clear objectives and a solution.
- No pressure for immediate implementation.
- Project requirements are clear and comprehensive.
- Project requirements are stable during the system development life cycle.
- When the project team are not so experienced or may change during implementation.

Variants of the Waterfall Model

Parallel Model

The Parallel Model is a variant of the traditional Waterfall model designed to reduce the long development time associated with strictly sequential phases. While it retains the structured and predictive nature of the Waterfall approach, it allows certain

development activities to be carried out simultaneously (in parallel) rather than one after the other.

In the Parallel Model, the overall system is divided into subprojects or modules after the general analysis and high-level design phases are completed. Each subproject then moves through the remaining phases, detailed design, implementation, and testing at the same time as other subprojects. Once all subprojects are completed, they are integrated into a single system.

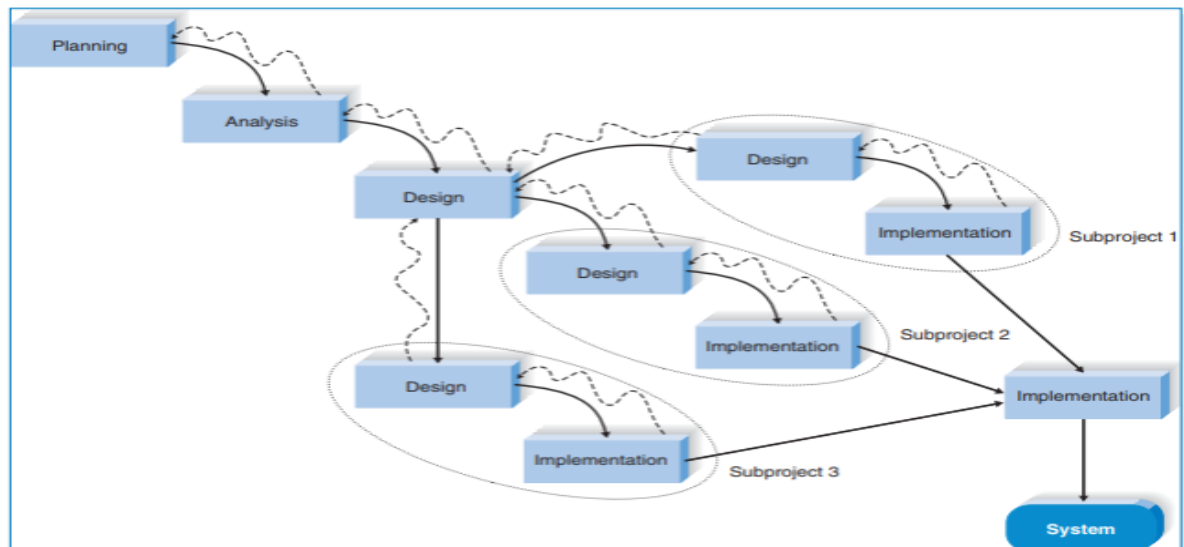


Figure 2: Parallel Model

V-Shape Model

V stands for verification and validation. The V-Shape Model (Verification and Validation Model) is a variant of the Waterfall model that emphasises testing at each development phase. It maintains a sequential structure but establishes a direct relationship between each development phase and its corresponding testing phase, as shown in Figure 3.

In the V-Shape Model, development activities proceed down the left side of the “V,” while testing activities move up the right side. For every development phase, a corresponding testing phase is planned in parallel. This ensures that verification and validation are built into the system development process from the beginning rather than being deferred until the end.

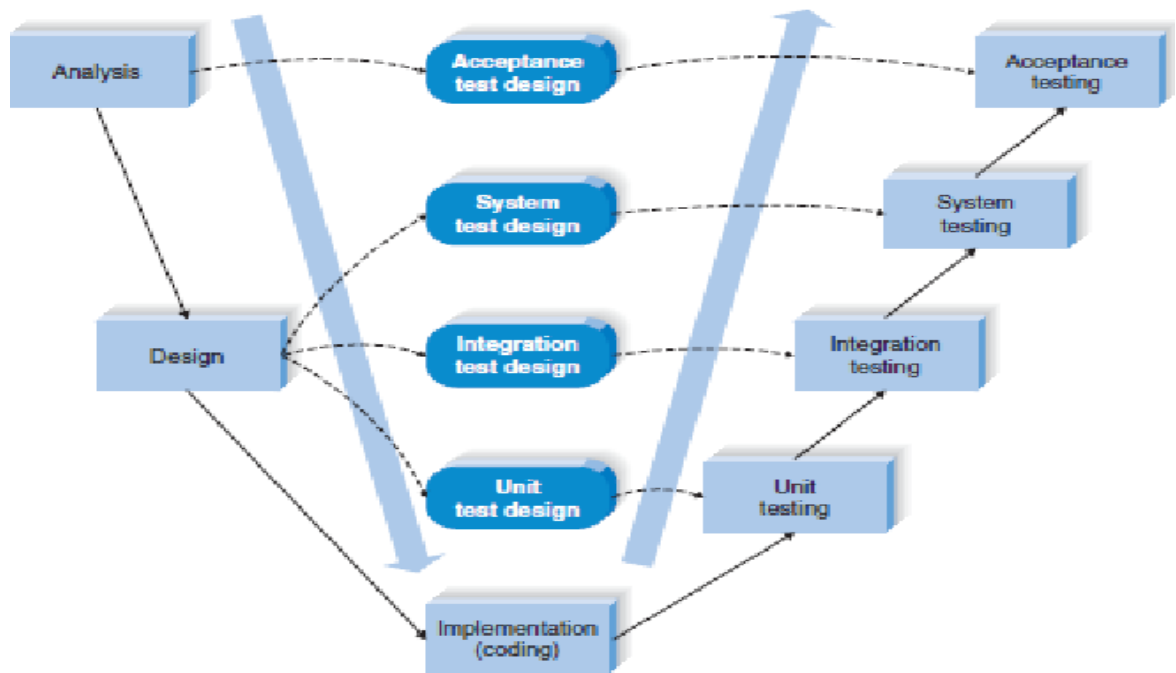


Figure 3: V-Shape

Advantages of the V-Shaped Model

1. It is simple and straightforward.
2. It improves the overall quality of the system through the emphasis on early development of test plans
3. Project management can track progress by milestones
4. There is a high quality of deliverables because of the test effectiveness
5. Product verification and validation is done in the early part of development

Disadvantages of the V-Shaped Model

1. It is also affected by the rigidity of the waterfall model
2. Does not easily handle dynamic changes in requirements
3. Does not suit the dynamic nature of the business environment
4. Does not contain risk analysis activities.

When to use the V-Shaped Model

1. When the systems to be built require high reliability, e.g hospital patient control applications.
2. When all the requirements are known up-front
3. Solution and technology to be used are known.

In-Text Questions (ITQs)

What type of project is a waterfall suitable for?

In-Text Answers (ITAs)

The Waterfall model is best suited for projects with stable and well-understood requirements, such as:

- Government systems,
- Financial applications, and
- Regulatory or safety-critical systems.

B. Rapid Application Development (RAD)

RAD (also called iterative development) uses specialised techniques and computer tools to speed up the analysis, design, and implementation phases, enabling a portion of the system to be developed quickly and delivered to users for evaluation and feedback. RAD emphasises quick development and delivery of systems through the use of reusable components, visual development tools, and intensive user involvement.

It also aims to develop and deliver a high-quality system at a relatively low investment cost. Reduces inherent project risk by breaking a project into smaller segments and providing greater ease of change during the development process. Delivers high-quality systems quickly, primarily using iterative prototyping involving active user involvement. It emphasises fulfilling business needs and produces documentation necessary to facilitate future development and maintenance.

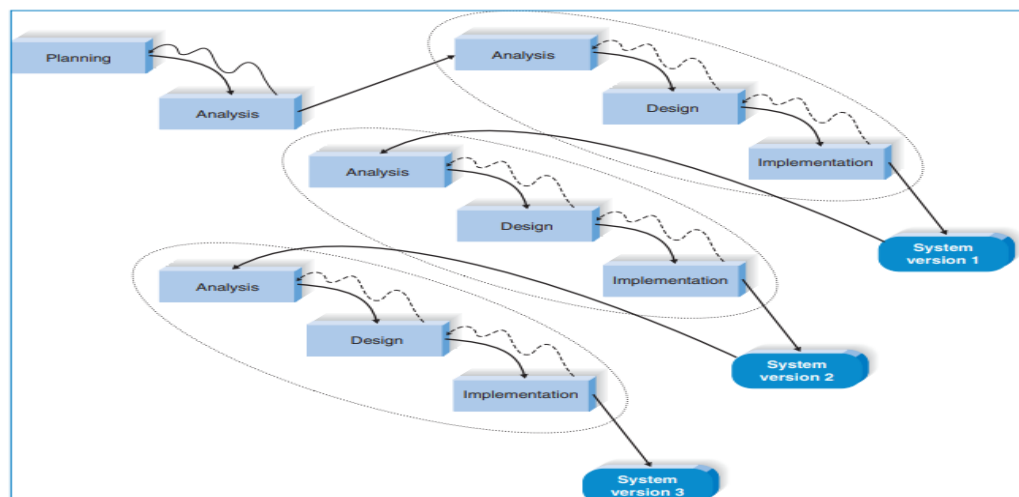


Figure 4: Rapid Application Development

The tools include:

- Graphical User Interface (GUI) builders,
- Computer-Aided Software Engineering (CASE) tools,
- Database Management System (DBMS),
- Fourth-generation programming languages, code generators, and object-oriented techniques.

RAD Advantages

1. Focus moves from documentation to code (WYSIWYG)
2. User feedback makes the model very effective
3. Time-box approach mitigates cost and schedule risk
4. Cost of development is lower as the system being built matches user expectations
5. Important additional requirements may be identified and incorporated into subsequent versions since users are working with the system.

Disadvantages of RAD

1. The chief disadvantage of iterative development is that users begin to work with a system that is intentionally incomplete.
2. More speed and lower costs may lead to lower overall system quality
3. User expectations may dramatically increase, and system requirements may expand during the project, leading to scope creep
4. Risk of never achieving closure, as user expectations cannot be totally managed
5. Developers and customers must be committed to rapid-fire activities in an abbreviated time frame

When to use Rapid Application

1. Reasonably well-known requirements
2. User will be involved throughout the life cycle
3. Low technical risks
4. High performance not required
5. Functionality delivered in increments

C. Incremental Model

The Incremental model develops the system in small, manageable portions called increments. Each increment adds new functionality and builds on previous releases. Combines linear and iterative system development methodologies with the aim of reducing inherent project risks by breaking a project into smaller segments. It is between waterfall and prototyping methods. Customers outline the services required of the system; these services are then prioritised and grouped into increments; each increment provides a subset of the system's functionality.

The increments are prioritised, and the first increment is defined in detail and developed. As subsequent increments are developed, they are integrated with existing increments; as such, the system functionality increases with each increment. Further requirements analysis can still be performed for later increments. However, changes for the current increment cannot be accepted.

Advantages of the Incremental Model

1. Develop high-risk or major functions first
2. Each release delivers an operational product
3. Lowers initial delivery cost
4. Uses "divide and conquer" breakdown of tasks
5. Initial product delivery is faster
6. Customers get important functionality early; they don't have to wait until the entire system is delivered before they can gain value from it
7. Risk of changing requirements and overall project failure is reduced

Disadvantages of the Incremental Model

1. It might be difficult to map customers' requirements to increments of the right size
2. Most systems require functionalities used by different parts of the system; requirements are not defined in detail until an increment is to be developed, and it might be difficult to identify common facilities needed by all requirements
3. Requires good planning and design
4. Requires early definition of a complete and fully functional system to allow for the definition of increments
5. Well-defined module interfaces are required (some will be developed long before others)

D. Prototyping Methodology

Prototyping is a type of development in which prototypes are developed early in the process to enable early feedback and analysis. A “quick and dirty” version of the system that contains minimal features is built. Following user reactions and comments, the developers reanalyse, redesign, and reimplement a second prototype that addresses deficiencies and adds more features. This cycle continues until the analysts, users, and sponsor agree that the prototype provides enough functionality to be installed and used in the organisation.

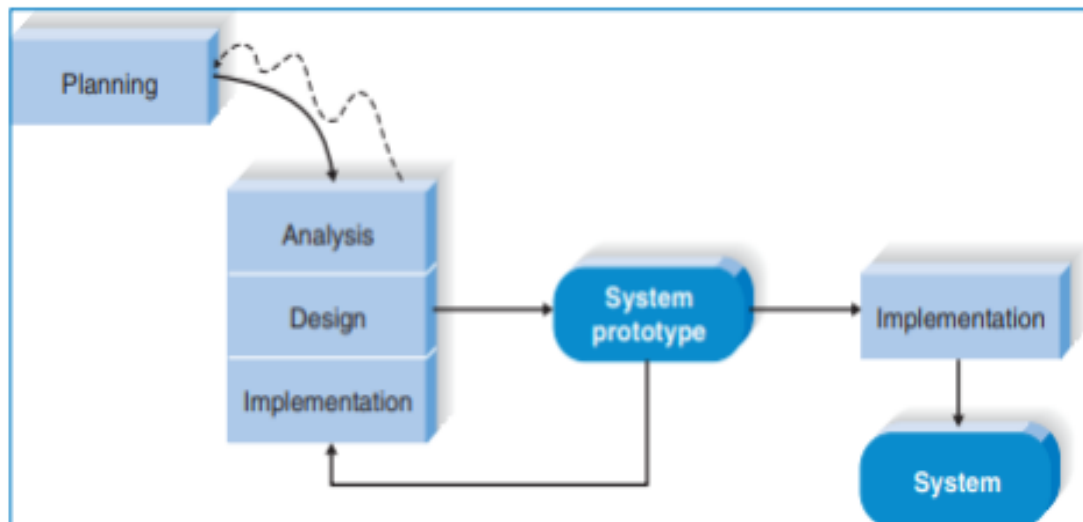


Figure 5: System Prototyping

There are different types of prototyping:

- Throw away prototyping
- Evolutionary prototyping

Evolutionary prototyping

Evolutionary prototyping involves building an initial version of the system and gradually refining and expanding it until it becomes the final operational system. The prototype evolves through continuous user feedback and iterative enhancements.

The objective of evolutionary prototyping is to deliver a working system to end-users. The development starts with those requirements that are best understood. Developers build a prototype during the requirements phase. The prototype is evaluated by end users. Users give corrective feedback. Developers further refine the prototype. When the user is satisfied, the prototype code is brought up to the standards needed for a final product.

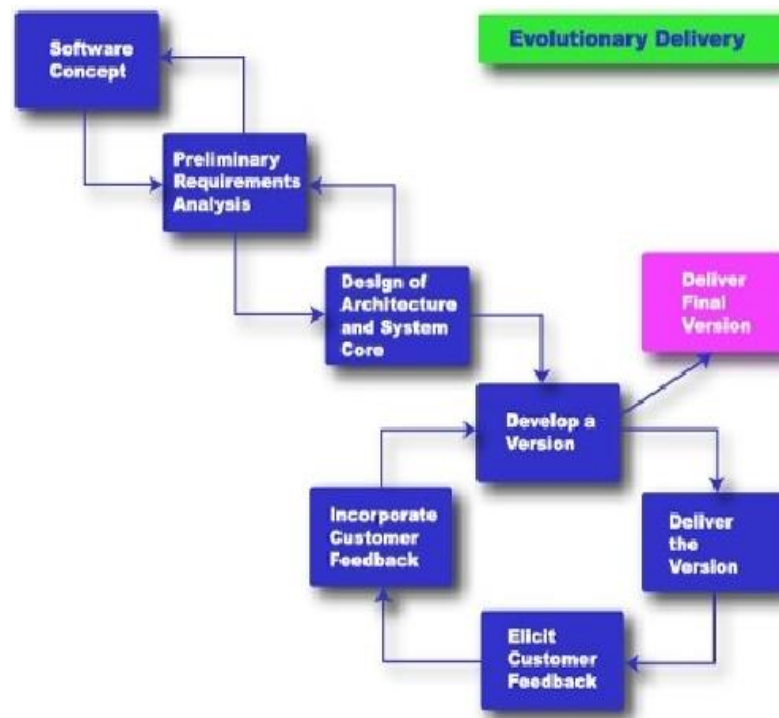


Figure 6: Evolutionary Prototyping

- A preliminary project plan is developed
- A partial high-level paper model is created
- The model is a source for partial requirements specification
- A prototype is built with basic and critical attributes
- The designer builds
 - the database
 - user interface
 - algorithmic functions
- The designer demonstrates the prototype; the user evaluates for problems and suggests improvements. This loop continues until the user is satisfied, and the prototype becomes the final system.

Advantages of Evolutionary Prototyping

- Customers can “see” the system requirements as they are being gathered
- A more accurate product is developed
- Unexpected requirements can be accommodated
- Developers learn from customers
- Allows for flexible design and development
- Interaction with the prototype stimulates awareness of additional needed functionality

Disadvantages of Evolutionary Prototyping

- Tendency to abandon structured program development for “code-and-fix” development
- lack of careful, methodical analysis prior to making design and implementation decisions
- Bad reputation for “quick-and-dirty” methods
- Overall maintainability may be overlooked
- The customer may want the prototype delivered
- Process may continue forever (scope creep)

When to use Evolutionary Prototyping

- Requirements are unstable or need to be clarified
- As the requirements clarification stage of a waterfall model
- Develop user interfaces
- Short-lived demonstrations
- New, original development
- With the analysis and design portions of object-oriented development

Throwaway Prototyping

Throwaway prototyping (also called rapid or exploratory prototyping) involves creating a temporary prototype to help users and analysts better understand system requirements. Here, prototypes are developed primarily to explore design alternatives rather than the actual new system. The prototype is built quickly and is discarded after requirements are clarified. The actual system is then developed using a more structured methodology.

There is a thorough analysis phase to gather requirements. However, some requirements may not be fully understood, or there may be technical issues to resolve. Design prototypes are then developed to help understand the issues under consideration. This helps reduce risk and produce more stable, reliable systems. It helps to refine key issues before the system is built.

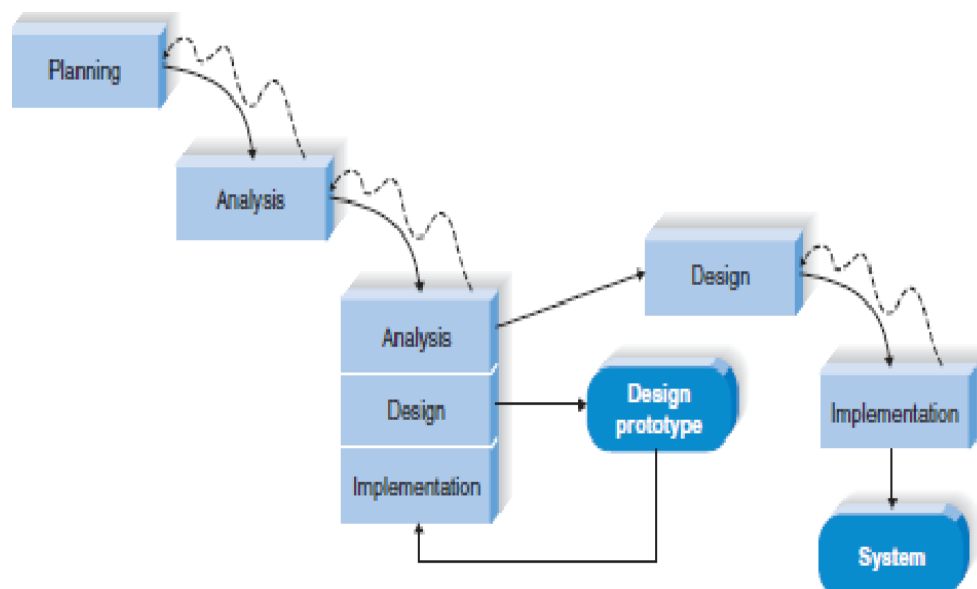


Figure 7: Throwaway prototyping

In-Text Questions (ITQs)

Distinguish between throwaway prototyping and evolutionary prototyping.

In-Text Answers (ITAs)

Throwaway Prototyping involves building a quick prototype solely to clarify requirements. Once understanding is achieved, the prototype is discarded, and the actual system is developed afresh.

It is used when requirements are unclear or users cannot articulate needs easily.

In evolutionary prototyping, the prototype is continuously refined and expanded until it becomes the final system. It is characterised by early system delivery, high user satisfaction, and flexibility to change. It stands the risk of weak architecture and difficult documentation.

It is used when requirements evolve, and rapid delivery is required

E. Agile Model

This model is a group of software development methods based on **adaptive**, **iterative**, and **incremental** development. Every iteration is a complete software project. Agile emphasises flexibility, collaboration, and the rapid delivery of working software. Unlike traditional models that rely on fixed requirements and sequential

phases, Agile recognises that requirements often change and therefore accommodates continuous refinement throughout the development process. Modelling and documentation overhead is eliminated, while face-to-face communication is preferred.

In the Agile model, system development is carried out in short, time-boxed cycles known as iterations or sprints. Each sprint typically includes activities such as requirement analysis, design, coding, testing, and review. At the end of each sprint, a working increment of the system is delivered for user evaluation and feedback.

Agile Activities include:

- Creation and prioritisation of user stories or requirements
- Sprint planning and task assignment
- Development and testing within each sprint
- Sprint review and retrospective for feedback and improvement

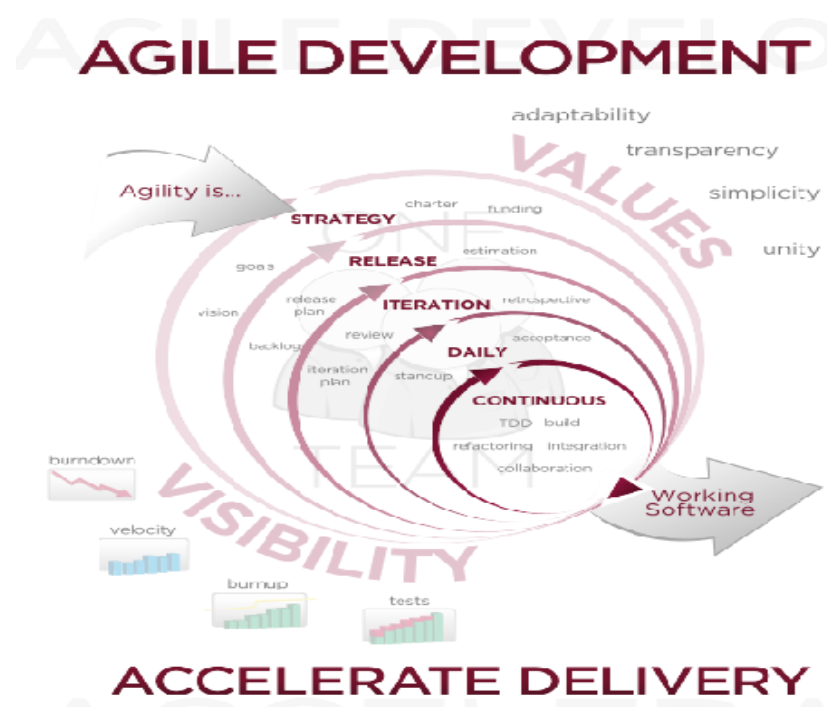


Figure 8: Agile Development

Popular agile methods include:

- Crystal Clear
- Scrum
- Extreme Programming (XP)
- Lean Software Development
- Kanban

1.3.2 Selecting the appropriate methodology

The first challenge for project managers is selecting the methodology to use. Choosing a methodology is not simple, because no one methodology is always best. Below are some criteria to use:

Clarity of User Requirements: When the user requirements for what the system should do are unclear, they are difficult to understand through discussion or written reports. System prototyping and throwaway prototyping are usually more appropriate when user requirements are unclear, because they provide prototypes for users to interact with early in the SDLC.

Familiarity with Technology: If the system is designed without familiarity with the base technology, risks increase because the tools may not be capable of doing what is needed. **Throwaway prototyping** is particularly appropriate for situations where there is a lack of familiarity with technology, because it explicitly encourages the developers to create design prototypes for areas with high risks

System Complexity: Complex systems require careful and detailed analysis and design. Throwaway prototyping is particularly well-suited to such detailed analysis and design, but system prototyping is not.

System Reliability: For some applications, reliability is truly critical (e.g., medical equipment, missile control systems), while for other applications it is merely important (e.g., games, Internet video). The **V-model** is useful when reliability is important, as it emphasizes testing. Throwaway prototyping is most appropriate when system reliability is a high priority, because detailed analysis and design phases are combined with the project team's ability to test many different approaches through design prototypes before completing the design.

Short Time Schedules: Projects with short schedules are well-suited for RAD methodologies because they are designed to accelerate development. Iterative development and system prototyping are excellent choices when timelines are short because they enable the project team to adjust system functionality to meet a specific delivery date.

Schedule Visibility: RAD methodologies move many critical design decisions earlier in the project to help project managers recognize and address risk factors and keep expectations in check.

Usefulness in Developing Systems	Waterfall	Parallel	V-Model	Iterative	System Prototyping	Throwaway Prototyping	Agile Development
with unclear user requirements	Poor	Poor	Poor	Good	Excellent	Excellent	Excellent
with unfamiliar technology	Poor	Poor	Poor	Good	Poor	Excellent	Poor
that are complex	Good	Good	Good	Good	Poor	Excellent	Poor
that are reliable	Good	Good	Excellent	Good	Poor	Excellent	Good
with short time schedule	Poor	Good	Poor	Excellent	Excellent	Good	Excellent
with schedule visibility	Poor	Poor	Poor	Excellent	Excellent	Good	Good

1.3.3 Conclusion

In conclusion, no single software development methodology is suitable for all projects. The success of system development largely depends on choosing a methodology that aligns with organizational goals, project characteristics, and stakeholder expectations. Traditional methodologies are effective for well-defined, stable projects, whereas adaptive approaches such as Agile and prototyping are better suited to dynamic environments with evolving requirements. By applying appropriate methodologies and adhering to best practices, developers can improve system quality, reduce development risks, and deliver information systems that deliver lasting business value.

Summary

In this unit, you have learned about Software development methodologies, which are a structured approach for planning, designing, building, testing, and maintaining information systems. The methodologies include predictive models such as Waterfall and its variants (Parallel and V-Shape) and adaptive models, such as Rapid Application Development (RAD), Prototyping (throwaway and evolutionary), Incremental models, and Agile methodologies. Each methodology defines how development activities are organized, how requirements are handled, and how users are involved. Traditional methodologies emphasize structure, documentation, and stable requirements, while adaptive methodologies focus on flexibility, iteration, and continuous user feedback.

Unit Activities

1. Read the content of Unit 1 in the study guide. Consult the recommended texts and research further online. (90 minutes)

Complete the self-introductory forum. Original post by Thursday. (10 minutes).

Complete replies to 2 of your classmates by Sunday at 11:59 p.m. (10 minutes)

2. TUTOR MARKED ASSIGNMENT: Complete the following and submit for grading. (60 minutes)

Tutor-Marked Assignment

You are the chosen project manager for a new clothing line about to be launched. You very much need the project to succeed and are willing to give it your all.

1. Outline and discuss five skills you must possess to make the project flourish.
2. Define the scope of the project, state activities to be conducted, and give a detailed outline of the timeline of your project (for the space of six months)
3. What are those criteria you will need to select good team members? List and explain 4 most important ones to you.
4. The requirements of this work **MAY NOT** be clearly stated at the beginning. Choose an appropriate SDLC methodology and map the project's phases.
5. You are nearing the end of the project, and the client is insisting on adding new features, but you want to prevent scope and feature creep. Do you refuse or agree to add the new features? Give three cogent reasons for your decision as PM.

N.B: Personalise the project. Use PP/slides. Send it to my email. Deadline is Monday, 11:59 pm.

References/Further Readings

System Analysis and Design, Alan Dennis, Barbara H. Wixom, and Robert M. Roth, John Wiley & Sons, 5th Edition, 2012.

System Analysis and Design Methods, Jeffery L. Whitten and Lonnie D. Bentley, 7th Edition, McGraw-Hill- Irwin, 2007

Object-Oriented Systems Analysis and Design Using UML, Simon Bennett, Steve McRob and Ray Farmer, McGraw-Hill, Second Edition, 2002.

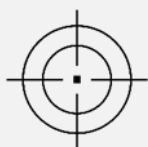
Module Two:

Requirements Modeling

Unit 1 — Fact Gathering Techniques

INTRODUCTION

Fact-gathering techniques form the foundation of a good systems analysis and design. They provide methods for systems analysts to collect accurate and relevant information about existing systems, user requirements, business processes, and organisational constraints. Without appropriate fact gathering, system development efforts risk being based on assumptions rather than real user needs and operational realities. This unit introduces the concept of fact-gathering techniques and explains their importance in understanding business problems and defining system requirements. It examines how analysts interact with users, documents, and operational environments to obtain the information required to support system development throughout the SDLC.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Define system requirements.
- Be able to differentiate between functional and non-functional requirements.
- Understand the concept of requirements discovery and management.
- Identify the seven fact-finding techniques and characterise the advantages and disadvantages of each.

2.1.1 Requirements Discovery

Requirements discovery is a major activity in systems analysis that focuses on identifying, understanding, and documenting what users and organisations need from an information system. Requirements discovery (RD) is the process and techniques used by systems analysts to identify and extract system problems (undesirable situations that prevent the organisation from fully achieving its mission, vision, goals, and objectives) and solution requirements from the user community. This process involves the system analyst and the users, working together. Requirements discovery requires analytical, communication, and interpersonal skills. Analysts must listen carefully, ask the right questions, resolve conflicting requirements, and validate findings with stakeholders to ensure accuracy and completeness.

Requirements discovery begins with an analysis of the current system or business process to identify problems and inefficiencies that necessitate the development of an improved or new information system. Systems analysts engage users, managers, and other stakeholders to understand how work is performed, what information is used, and what outcomes are expected.

In the process, two types of requirements are identified: functional and non-functional. A variety of fact-gathering techniques are used during requirements discovery. Common techniques include interviews, questionnaires, workplace user observation, document analysis, and facilitated workshops or Joint Application Development (JAD) sessions. In some cases, prototyping is also used to help users visualise potential system features and clarify their requirements through feedback and iteration.

The process of requirements discovery consists of four activities:

1. Problem discovery and analysis
2. Requirements discovery
3. Documenting and analysing requirements
4. Requirements management

2.1.2 System requirements

System requirements are statements that specify what an information system should do and the conditions under which it must operate. They define the expected functions, behaviour, and constraints of a system and serve as a formal agreement between stakeholders and the development team. System requirements are developed

during the system analysis phase and provide the basis for system design, implementation, testing, and evaluation.

In simple terms, system requirements are what the information system must do (functional requirements) or what it must have (non-functional requirements). It is also called a **business requirement**. System requirements translate user needs and business objectives into testable specifications. They ensure that the system addresses organisational problems, supports business processes, and delivers measurable value.

Results of Incorrect Requirements

- The system may cost more than projected.
- The system may be delivered later than promised.
- The system may not meet the users' expectations, and that dissatisfaction may cause them not to use it.
- Once in production, the costs of maintaining and enhancing the system may be excessively high.
- The system may be unreliable, prone to errors, and downtime.
- The reputation of the IT staff on the team is tarnished because any failure, regardless of who is at fault, will be perceived as a mistake by the team.

2.1.3 Fact-finding

Fact-finding is the formal process of using research, meetings, interviews, questionnaires, sampling, and other techniques to collect information about system problems, requirements, and preferences. It is also called information gathering or **data collection**.

Fact-Finding Ethics

Fact-Finding often brings systems analysts into contact with sensitive information such as:

- Company plans
- Employee salaries or medical history
- Customer credit card, social security, or other information

The analyst's ethical behaviour includes:

- Systems analysts must not misuse that information.
- Systems analysts must protect that information from people who would misuse it.

Otherwise:

- Systems analyst loses respect, credibility, and confidence of users and management, impairing the ability to do the job
- An organisation and systems analyst could have legal liability
- A systems analyst could lose a job

Documenting and Analysing Requirements

Documenting requirements involves formally recording them in a clear, unambiguous manner. The goal is to create a shared understanding among stakeholders, that is, analysts, designers, developers, users, and sponsors.

A requirements definition document provides a high-level description of system functions and constraints, written in business language.

Requirements Definition Document – A formal document that communicates the requirements of a proposed system to key stakeholders and serves as a contract for the system's project.

The draft requirements are documented with various tools:

- **Use cases:** Use cases describe how users (actors) interact with an information system to achieve specific goals. They focus on what the system does from the user's perspective rather than how it is implemented. Use cases help analysts understand functional requirements by clearly defining system behaviour in real-world scenarios.
- **Decision tables:** Decision tables are used to represent complex business rules and decision logic. They list all possible conditions and corresponding actions in a tabular format, ensuring that every possible scenario is considered. Decision tables are useful when multiple conditions affect system behaviour. They help eliminate ambiguity, ensure completeness, and support accurate implementation of business rules in the system.
- **Requirements tables:** Requirements tables organise system requirements in a tabular form, including details such as requirement ID, description, type (functional or non-functional), source, priority, and status.

Requirements tables help analysts track and validate requirements throughout the system development life cycle. It makes it easier to monitor changes and ensure that all requirements are addressed during design, development, and testing.

The requirement definition document is often also referred to as:

- Requirements definition report

- Requirements statement
- Requirements specification
- Functional specifications

In-text Question (ITQ)

Why is it important to ensure correct system requirements at the beginning of the system development process?

In-text Answer (ITAs)

System requirements are requirements that the information system must meet (functional requirements) or properties it must have (non-functional requirements). Incorrect requirements may result in any of the following

- cost overruns.
- System delivered later than promised.
- system not meeting the users' expectations, and that dissatisfaction may cause them not to use it.
- high costs of maintaining and enhancing the system.
- The system may be unreliable, prone to errors, and downtime.
- Loss of reputation of the IT team

2.1.4 Fact-Finding Methods

There are seven fact-finding techniques, namely.

- Sampling of existing documentation, forms, and databases.
- Research and site visits.
- Observation of the work environment.
- Questionnaires.
- Interviews.
- Prototyping.
- Joint requirements planning (JRP).

Sampling of existing documentation, forms, and databases.

This technique involves reviewing existing organisational documents, such as:

- Organisation chart
- Memos and other documents that describe the problem
- Standard operating procedures for the current system
- Completed forms
- Manual and computerised screens and reports
- Samples of databases
- Flowcharts and other system documentation

Sampling existing documents helps the systems analyst to gain insight into current business processes, business rules, and other information requirements. It also helps identify redundancies and inconsistencies in processes and recordkeeping, as well as data quality issues that the information system needs to address.

Observation of the Work Environment

Observation is a fact-finding technique in which the systems analyst either participates in or observes a person performing activities to learn about the system. This technique helps uncover informal procedures, workarounds, bottlenecks, and inefficiencies that users may not mention during interviews. Observation provides a first-hand understanding of how work is done, knowledge of user behaviour, and system usage.

Observation Guidelines

To make observation effective, the following guide can be followed:

- Determine the who, what, where, when, why, and how of the observation.
- Obtain permission from appropriate supervisors or managers.
- Inform those who will be observed of the purpose of the observation.
- Keep a low profile.
- Take notes during or immediately following the observation.
- Review observation notes with appropriate individuals.
- Don't interrupt the individuals at work.
- Don't focus heavily on trivial activities.
- Don't make assumptions.

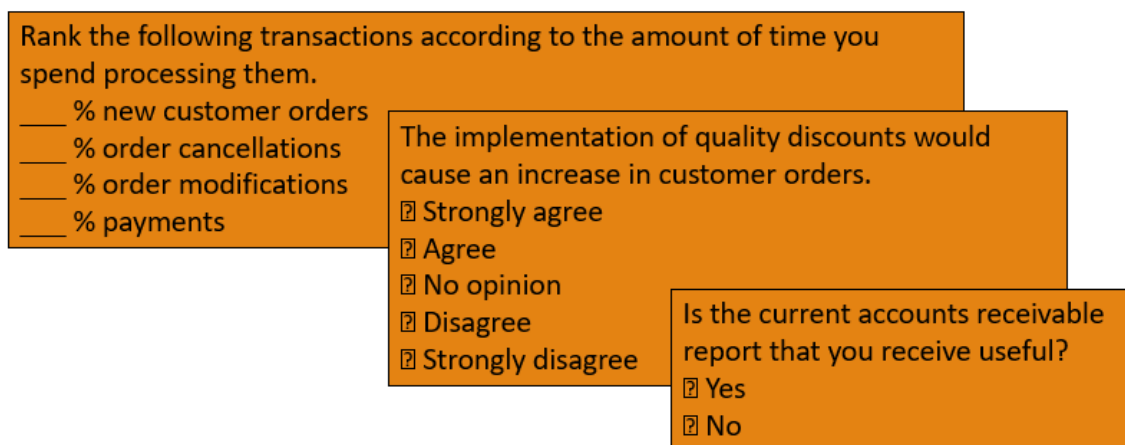
Questionnaires

A questionnaire is a special-purpose document that allows the analyst to collect information and opinions from respondents. They are useful when information is

needed from many people quickly and at low cost. Questionnaires can include open-ended (free-format) or closed-ended (fixed-format) questions.

Free-format questionnaire – A questionnaire designed to offer the respondent greater latitude in the answer. A question is asked, and the respondent records the answer in the space provided after the question.

Fixed-format questionnaire – A questionnaire containing questions that require selecting an answer from predefined available responses. Types of fixed-format questions include multiple-choice, rating, and ranking questions. Figure 1 shows examples of fixed-format questions



Rank the following transactions according to the amount of time you spend processing them.

% new customer orders
 % order cancellations
 % order modifications
 % payments

The implementation of quality discounts would cause an increase in customer orders.

☐ Strongly agree
☐ Agree
☐ No opinion
☐ Disagree
☐ Strongly disagree

Is the current accounts receivable report that you receive useful?

☐ Yes
☐ No

Figure 1: Example of fixed-format questions

Interviews

A fact-finding technique whereby the systems analysts collect information from individuals through face-to-face interaction. It can be used to:

- Find facts
- Verify facts
- Clarify facts
- Generate enthusiasm
- Get the end-user involved
- Identify requirements
- Solicit ideas and opinions

Types of Interviews and Questions are:

Unstructured interview – An interview that is conducted with only a general goal or subject in mind and with few, if any, specific questions. The interviewer counts on the interviewee to provide a framework and direct the conversation.

Structured interview – An interview in which the interviewer has a specific set of questions to ask of the interviewee. In a structured interview, there are two types of questions that may be asked:

- Open-ended question – A question that allows the interviewee to respond in any way that seems appropriate.
- Closed-ended question – A question that restricts answers to either specific choices or short, direct responses.

Procedure to Conduct an Interview

Conducting a successful interview requires planning and structure to gather the needed information. The interview procedure usually involves the following stages:

- Define the objective of the interview: First, identify the goals of the interview. This includes determining what information is needed, which processes or systems are being examined, and how the information will be used. Clear objectives are required to make the interview as focused and productive as possible.
- Select Interviewees: identify the most appropriate end users to interview. Learn about the individual prior to the interview
- Prepare for the Interview: An interview guide is a checklist of specific questions the interviewer will ask the interviewee. Figure 2 shows a sample interview guide.
- Conduct the Interview: Establish the purpose of the interview, ask questions in a professional manner, and practice active listening. Stay neutral when you hear opinions and encourage honest, open responses. Take notes or ask the interviewee permission to record. The following summarises the guidelines for Communicating
 - Approach the Session with a Positive Attitude
 - Set the Other Person at Ease
 - Let Them Know You Are Listening
 - Ask Questions
 - Don't Assume Anything
 - Take Notes
 - **Body language** – The non-verbal information we communicate.
 - Facial disclosure
 - Eye contact
 - Posture

- Follow Up on the Interview: validate the documented responses during the interview with the interviewee or relevant stakeholders to ensure correctness and avoid misunderstandings. This confirmation enhances the reliability of the gathered information.

Interviewee: Jeff Bentley, Accounts Receivable Manager Date: January 19, 2003 Time: 1:30 P.M. Place: Room 223, Admin. Bldg. Subject: Current Credit-Checking Policy		
Time Allocated	Interviewer Question or Objective	Interviewee Response
1 to 2 min.	Objective Open the interview: • Introduce ourselves • Thank Mr. Bentley for his valuable time. • State the purpose of the interview — to obtain an understanding of the existing credit-checking policies.	
5 min.	Question 1 What conditions determine whether a customer's order is approved for credit? Follow-up	
5 min.	Question 2 What are the possible decisions or actions that might be taken once these conditions have been evaluated? Follow-up	
3 min.	Question 3 How are customers notified when credit is not approved for their order? Follow-up	
1 min.	Question 4 After a new order is approved for credit and placed in the file containing orders that can be filled, a customer might request that a modification be made to the order. Would the order have to go through credit approval again if the new total order cost exceeds the original cost? Follow-up	

Figure 2: Sample Interview Guide

Discovery Prototyping

The act of building a small-scale, representative, or working model of the users' requirements to discover or verify those requirements. Users interact with the prototype and provide feedback, which helps clarify requirements and identify misunderstandings. Prototyping is particularly useful when requirements are unclear or when user interface design is critical to system success.

Joint Requirements Planning

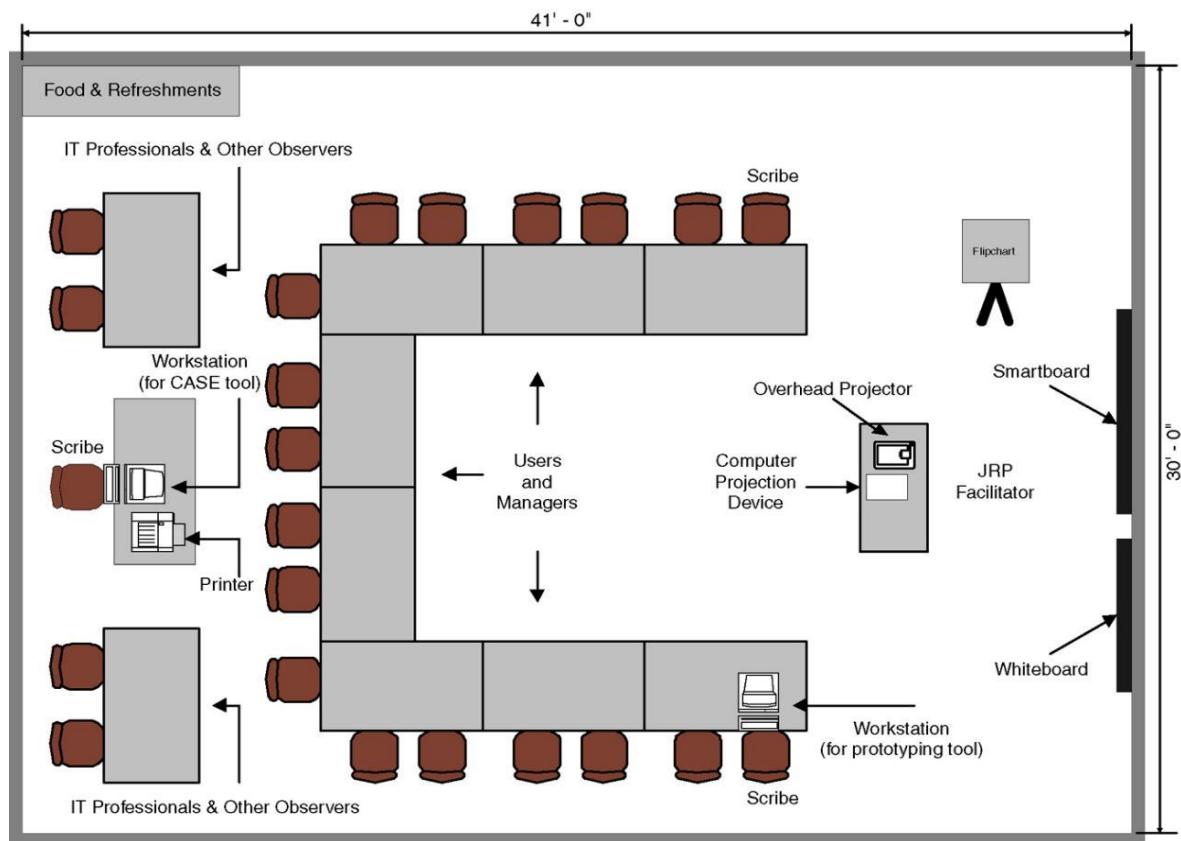
A process whereby highly structured group meetings are conducted for the purpose of analysing problems and defining requirements. JRP is a subset of a more comprehensive joint application development (JAD) technique that encompasses the entire systems development process. JRP Participants are:

- Sponsor
- Facilitator
- Users and Managers
- Scribes (Note takers)

- IT Staff

Steps to Plan a JRP Session

1. Selecting a location
 - Away from the workplace when possible
 - Requires several rooms
 - Equipped with tables, chairs, a whiteboard, overhead projectors, and
 - Needed computer equipment. See Figure 3
2. Selecting the participants: Each of the participants needs to be released from regular duties
3. Preparing the agenda: Briefing documentation distributed before each session



Guidelines for successful JRP Session

- Do not unreasonably deviate from the agenda
- Stay on schedule
- Ensure that the scribe can take notes
- Avoid the use of technical jargon
- Apply conflict resolution skills
- Allow for ample breaks

- Encourage group consensus
- Encourage user and management participation without allowing individuals to dominate the session
- Make sure that attendees abide by the established ground rules for the session

One goal of a JRP session is to generate possible solutions to a problem. Brainstorming is a common approach that is used for this purpose.

Brainstorming – A technique for generating ideas by encouraging participants to offer as many ideas as possible in a short period of time, without any analysis until all the ideas have been exhausted.

Brainstorming Guidelines

- Isolate the appropriate people in a place that will be free from distractions and interruptions.
- Make sure everyone understands the purpose of the meeting.
- Appoint one person to record ideas.
- Remind everyone of brainstorming rules.
- Within a specified period, team members call out their ideas as quickly as they can think of them.
- After the group has run out of ideas and all ideas have been recorded, then and only then should the ideas be analysed and evaluated.
- Refine, combine, and improve the ideas that were generated earlier.

Benefits of JRP

- JRP actively involves users and management in the development project (encouraging them to take “ownership” in the project).
- JRP reduces the amount of time required to develop systems.
- When JRP incorporates prototyping as a means for confirming requirements and obtaining design approvals, the benefits of prototyping are realised.

In-Text Question (ITQs)

1. List the fact-finding techniques.
2. What fact-finding techniques involve getting ideas from different stakeholders?

In-Text Answers (ITAs)

1. These are fact-finding techniques.
 - i. Sampling of existing documentation, forms, and databases.
 - ii. Research and site visits.
 - iii. Observation of the work environment.
 - iv. Questionnaires.
 - v. Interviews.
 - vi. Prototyping.
 - vii. Joint requirements planning (JRP).
2. Joint Requirement Planning. It involves brainstorming, which is a technique for generating ideas by encouraging participants to offer as many ideas as possible in a short period of time, without any analysis until all the ideas have been exhausted.

2.1.5 Conclusion

Requirements discovery is achieved using appropriate fact-finding techniques. Well-analysed requirements form the foundation for system design, implementation, and testing. It reduces the risk of errors, rework, cost overruns, and system failure. By applying appropriate fact-finding and actively involving stakeholders, systems analysts can ensure that the systems they develop align with organizational goals, support business processes, and deliver lasting value.

Summary

In this unit, you have learned about requirements discovery and fact-finding techniques, which are fundamental activities in systems analysis and design. Requirements discovery focuses on identifying and understanding what users of an information system need. It is divided into functional and non-functional requirements. It ensures that system development is driven by solving business problems and meeting users' expectations, rather than by assumptions.

Fact-finding techniques provide methods for gathering the information needed during requirements discovery. Techniques such as sampling existing documents and databases, research and site visits, observation, questionnaires, interviews, prototyping, and Joint Requirements Planning (JRP) are used. A combination of these methods enables systems analysts to collect comprehensive and reliable data. Each technique has unique strengths and limitations, so the analysts often combine

multiple techniques to gain an accurate understanding of the current system and user needs.

Self-Assessment Questions

1. Differentiate between functional and non-functional requirements
2. Highlight the processes involved in conducting an interview
3. Is joint requirement planning (JRP) sufficient to be used as the only fact-finding technique in system development? Support your answer with reasons.

Unit Activities

1. Read the content of Unit 1 in the study guide. Consult the recommended texts and research further online. (90 minutes)
2. Complete the self-introductory forum. Original post by Thursday. (10 minutes). Complete replies to 2 of your classmates by Sunday at 11:59 p.m. (10 minutes)

Tutor-Marked Assignment

1. Complete the following and submit for grading. (60 minutes)

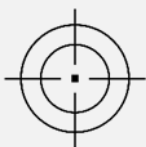
References/Further Readings

- System Analysis and Design, Alan Dennis, Barbara H. Wixom, and Robert M. Roth, John Wiley & Sons, 5th Edition, 2012.
- System Analysis and Design Methods, Jeffery L. Whitten and Lonnie D. Bentley, 7th Edition, McGraw-Hill- Irwin, 2007
- Object-Oriented Systems Analysis and Design Using UML, Simon Bennett, Steve McRob and Ray farmer, McGraw-Hill, Second Edition, 2002.
- Software Engineering, Ian Sommerville (2004)7th Edition. Chapter 6.

Unit 2 — Modelling System Requirement with use Cases

INTRODUCTION

In this unit, you will be introduced to one of the techniques for modelling system requirements: use cases. Use cases focus on the interactions between users and the system, describing system behaviour from the user's perspective rather than from a technical or implementation viewpoint. This unit introduces the concept of modelling system requirements with use cases and explains their role in translating user requirements into well-defined functional requirements. It examines how use cases identify system actors, define system boundaries, and capture interaction scenarios.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Explain the importance of use cases in the analysis phase of the SDLC.
- Describe the various parts of a use case and the purpose of each part.
- Create a use case to model a functional requirement.

2.2.1 USE CASES MODELING

Use cases help clarify users' interactions with the system and can reveal the new system's functional requirements. Use case diagrams are widely used in the analysis phase to involve users in the requirements elicitation process. "A use case represents how a system interacts with its environment by illustrating the activities that are performed by the users of the system and the system's responses." (Dennis, Wixon & Roth). Analysts usually create use cases to describe all the tasks that users need to perform with the system. Use cases are regarded as an external or functional perspective on a business process, showing how users interact with it rather than its internal procedures or technical details.

Importance of Use Case in Analysis and Design

- Use cases help to identify and clarify details about the functional requirements of the new system. Most system developers agreed that a well-developed use case diagram captures most of the system's functional requirements.
- Use cases are very helpful in understanding exceptions, special cases, and error handling requirements in the new system. These are easily overlooked by developers, but use cases help to discover them.
- The text-based use case that is the use case narrative is easy for the users to understand, but it also flows easily into the creation of process models (Unit 3) and the data model (Unit 4).

Use-case modelling is the process of modelling a system's functions in terms of business events, who initiates them, and how the system responds to them.

Two primary outputs or products are involved when performing use case modelling:

- i. the use case diagram and
- ii. the use case narrative

Use-case diagram

A use-case diagram graphically depicts the system as a collection of use cases, actors (users), and their relationships. It depicts the interactions between the system, external systems, and the user. The diagram provides a high-level view of the business events the system must process. It graphically depicts who will use the system and how users expect to interact with it. Use cases describe the system's functions from the perspectives of external users, in a manner and terminology they understand. Figure 1 depicts a simple use case diagram.

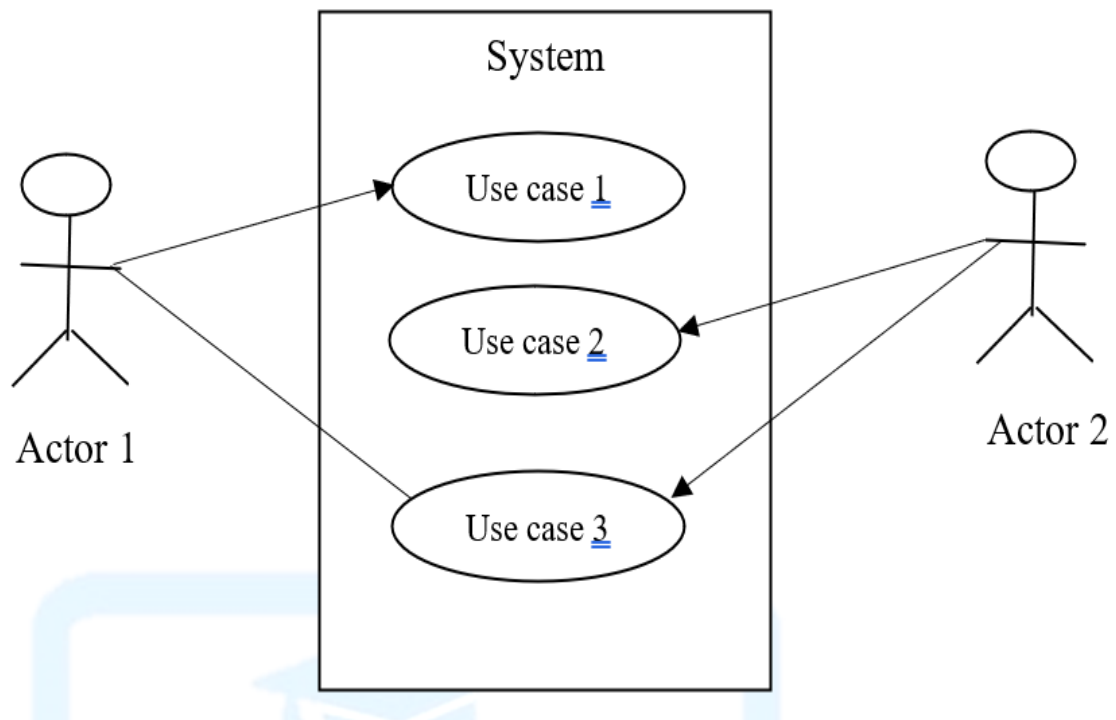


Figure 1: Sample use case diagram

Elements of Use Diagram

Use cases: Use cases describe the system functions from the perspectives of external users and in a manner and terminology they understand. A use case documents the interaction required between the user and the system to accomplish the user's task. Use cases can also be referred to as system activities. Use cases are created to help the development team better understand the steps required to achieve the user's goals. Once created, use cases can often be used to derive more detailed functional requirements for the new system. Use cases are represented by Eclipse with the name written inside.



Actors: Use cases are triggered by external users called actors. An actor is a use case for completing a business task that produces something of measurable value. For example, in a course registration system, the student is an actor, and the use case or business event is the registration of a course or course registration. Actors are always outside the system boundary and initiate or participate in use cases by sending inputs to the system or receiving outputs from it. Each actor is defined based on what they do in relation to the system.



Actor Symbol

There are primarily four types of actors:

- i. **Primary business actor:** This actor primarily benefits from the execution of the use case by receiving something of measurable or observable value. For e.g., an employee receiving a salary from a firm.
- ii. **Primary system actor:** This actor directly interfaces with the system to initiate or trigger a business event. For example, the bursar who dispenses cash to the employee.
- iii. **External server actor:** This actor responds to a request from the use case. For e.g., the Financial Director approving cash dispensing to an employee
- iv. **External receiver actor:** This actor is not the primary actor but receives something of measurable or observable value (output) from the use case. For e.g., a warehouse receiving a packing order to prepare a shipment after an employee has placed an order.

Relationships: Relationships in use case diagrams define how actors and use cases are connected and how different use cases interact with one another. Relationships show the system's behaviour, including reuse functionality and dependencies among system functions. The main types of relationships in use case modelling are:

- i. **Association (Communication Relationship):** An association represents a direct interaction between an actor and a use case. It shows that the actor participates in or initiates the use case. This relationship is typically drawn as a simple line connecting the actor to the use case. It has two types:
 - Associations with arrowhead: an arrowhead touching the use case indicates that the use case was initiated by the actor on the other end of the line.
 - Associations without arrowhead: indicate an interaction between the use case and an external server or receiver actor.
- ii. **Extends Relationship (<<extends>>):** Sometimes, a use case may contain complex functionalities consisting of several steps, making the use-case logic

difficult to understand. An extension will extract these complex functionalities into their own use case, allowing users to better follow the process. For instance, in Figure 2, "Generate Warehouse Packing Order" and "Calculate Order Subtotal & Sales Tax" use cases are extensions of the "Place New Member Order" use case.

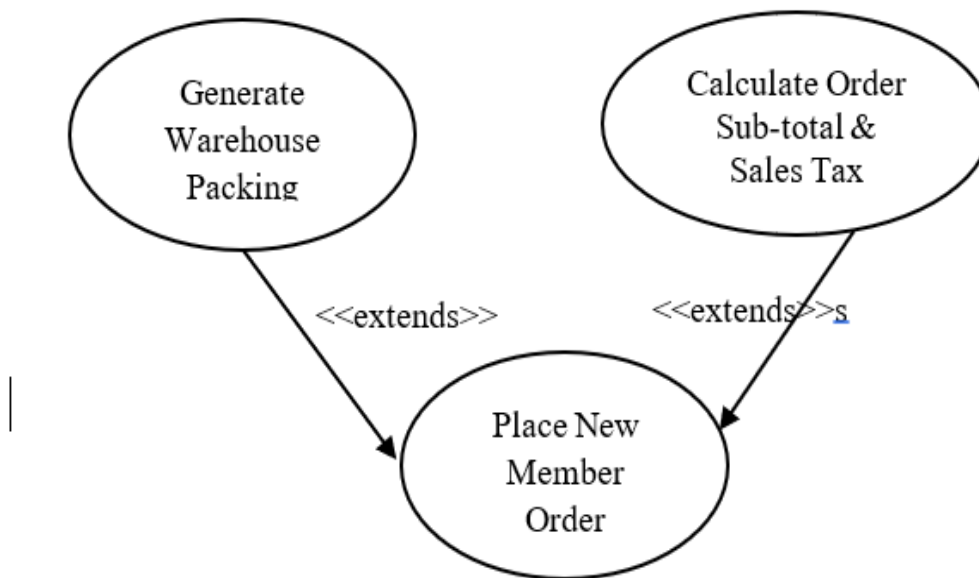


Figure 2: Extend Use Case

The extended relationship is an optional or conditional behaviour that can extend the functionality of a base use case under specific conditions

Note: The arrow is drawn from the extension used to the use case that is being extended.

iii. Include or Use Relationship(<<include>>)

The «include» relationship is used when one use case always requires the behaviour of another use case as part of its execution. The included use case represents common functionality that is reused by multiple use cases. Including use cases helps promote reuse and avoid the duplication of functionality. For example, whenever a customer needs to place an order, the customer needs to be validated. Also, the customer may need to be validated to make a payment.

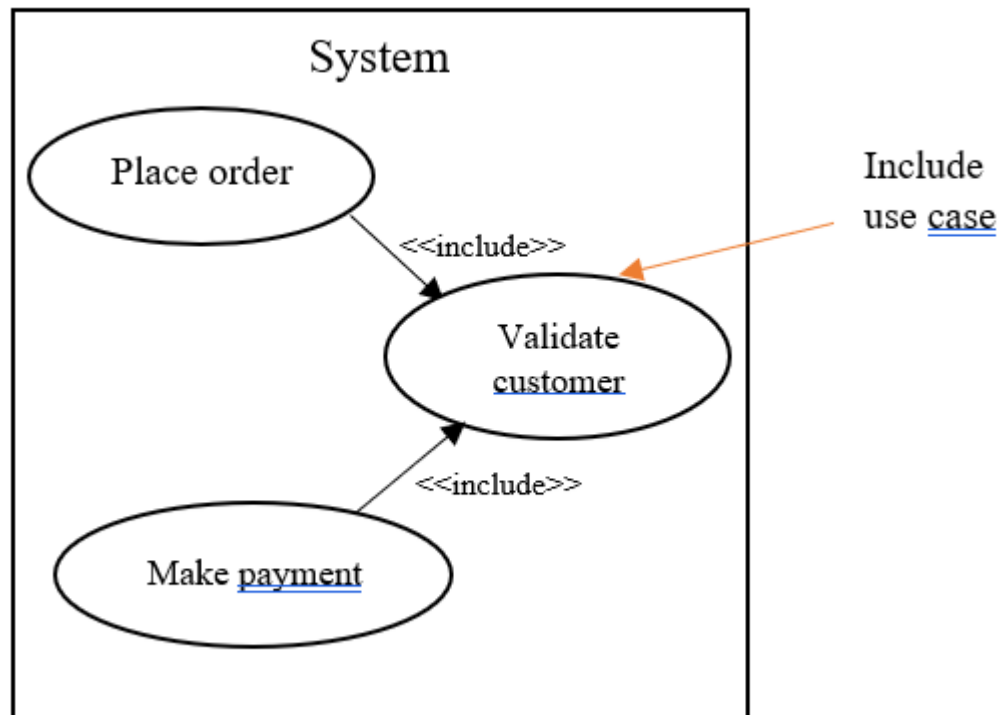


Figure 3: Include or Uses Relationship

- iv. Depends on relationship: this is a relationship in a use case that indicates that one use case relies on another use case for its execution. For instance, in a bank, 'withdrawing cash' depends on first 'depositing cash'. Likewise, 'depositing cash' depends on 'opening a bank account'. It indicates that one use case relies on another, but it is not necessarily a direct association or inclusion. In this relationship, a change in one use case may affect another, making it valuable for managing change impact.

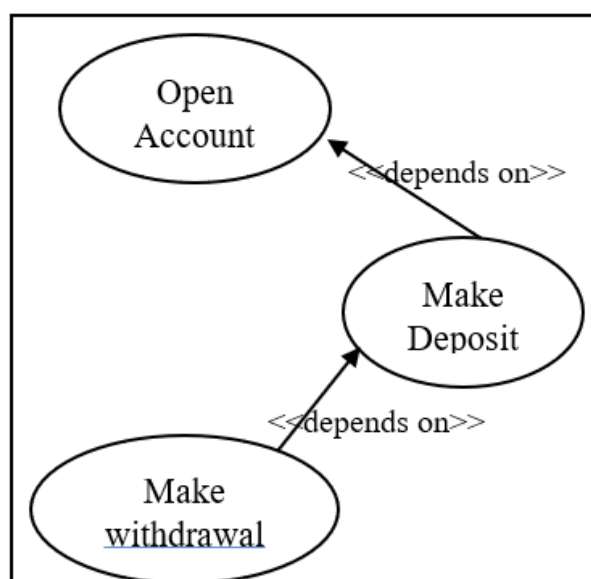


Figure 4: Depends on Relationship

- v. Generalisation (Inheritance) relationship: When two or more actors have the same function or behaviour, that is, they are able to initiate the same use case, it is advisable to extract this shared function or behaviour and represent it through a new abstract actor. This approach helps reduce redundant interactions and simplifies communication between the actors and the system. This is also referred to as inheritance. For example, in a bank, the Customer actor represents any person who uses the banking system and can perform activities such as withdrawing and depositing money. Customers can also be categorised based on account type, such as savings and current accounts. The Savings Account Holder and Current Account Holder are specific types of customers who inherit all use cases associated with Customer (e.g., Withdraw Money, Check Balance), but may also have additional specific use cases. Generalisation is denoted with a triangle arrowhead as shown in Figure 5

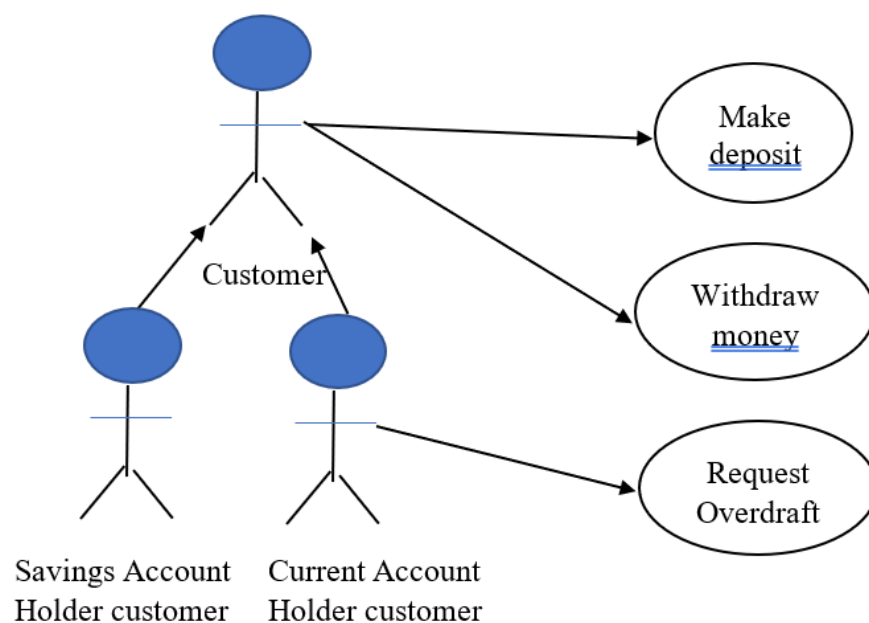


Figure 5: Use case showing inheritance

System Boundary: The system boundary represents the scope of the system in use and includes the name of the system inside or on top of the use case diagram.

It is a rectangle used to separate the system's internal use cases from external actors. See Figure 6 for a sample of a complete use case diagram.

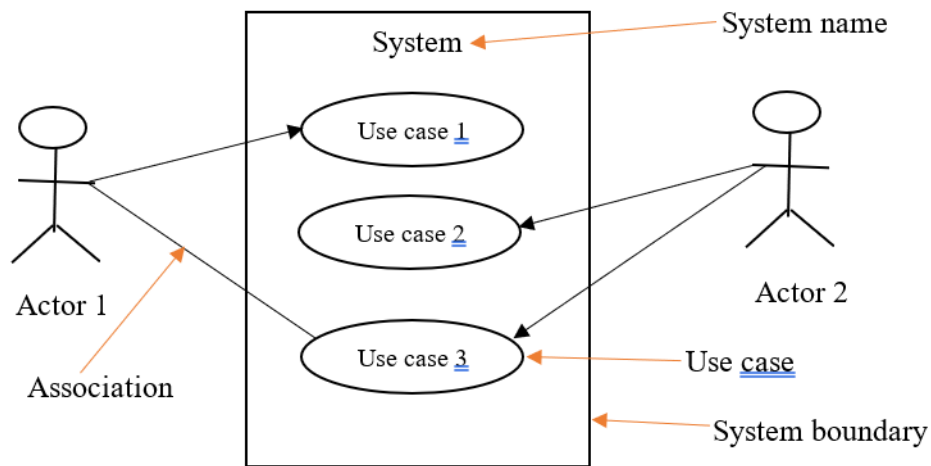


Figure 6: Component of a Use Case Diagram

Use-case narrative

A use-case narrative is the textual description of a use-case. It describes how an actor interacts with a system to achieve a specific goal. It explains the sequence of events that occur between the user and the system step by step. This provides a clear understanding of system behaviour without focusing on technical implementation details.

A use-case narrative contains the name of the use case, the primary actor, a brief description of the goal, and the main flow of events, also known as the basic course of action. It may also include alternative flows and exception flows, which describe variations in user actions or system responses when conditions differ, or errors occur.

Basic Elements of a Use Case Narrative

Use Case Name and Unique Identifier: Name is a short text that describes the action being performed in the system. Example: Withdraw Money, Check Balance. The identifier is a unique number used to reference the use case.

Actor(s): Actors are the people, systems, or external entities that interact with the system.

Description / Goal: This explains what the actor wants to achieve by performing the use case. It provides a summary of the use case's purpose and a quick understanding of its objective.

Example: Allows the customer to withdraw cash from their account.

Preconditions: Conditions that must be true before the use case begins. If these conditions are not met, the use case cannot proceed.

For example, to withdraw money, a customer must have a valid ATM card and enter a correct PIN.

Main Flow (Basic Flow): This lists the step-by-step sequence of actions between the actor and the system. It describes the ideal scenario in which everything works correctly, showing how the use case operates from start to finish. It is usually numbered for clarity.

Alternative Flows (Extensions): Describes variations, exceptions, or error situations that may occur. It illustrates a real-world scenario in which things may not go as planned, often referred to as a branch off the main flow.

For example, in a cash withdrawal use case, the PIN may be invalid, or there may be insufficient funds in the account.

Postconditions: Describes what must be true after the use case is completed. It shows the system's final state and confirms the outcome of the use case.

Example: Account balance updated

In-Text Questions (ITQs)

Describe the relationships in the use case

In-Text Answers (ITAs)

- a) Association: is a relationship between an actor and a use case that exists whenever the use case describes an interaction between them.
- b) Uses (or includes): When two or more cases perform identical functionalities. These common functionalities are abstracted into a separate use case called an abstract use case.
- c) Depends on: This is a relationship between use cases, indicating that one use case cannot be performed until another use case has been performed.
- d) Inheritance (Generalisation): is used when one actor (and/or use case sometimes) is a specialised version of another. The child actor or use case inherits the behaviour and relationships of the parent.

2.2.2 Requirements Modelling

Requirements use-case modelling is a structured approach for identifying, analysing, and representing system requirements through use cases. It focuses on capturing how users interact with a system to achieve specific goals.

The system can be modelled “as-is” or “to be”

Modelling a system “As-Is” refers to documenting and representing the current state of an existing system before changes or improvements are made. It focuses on how the system currently operates, including its processes, data flows, rules, users, and technologies. Modelling “as-is” is good for gaining a clear and accurate understanding of the existing environment without introducing assumptions or proposed solutions.

Modelling the “To-Be” system involves representing the desired future state of a system after improvements or a new information system have been implemented. It describes how the system should operate to better support business goals.

The steps required to produce this model are:

1. Identify business actors.
2. Identify business requirements and use cases.
3. Construct a use-case model diagram.
4. Document business requirement use-case narrative

Identify business actors.

Actors are users of the system. Actors indicate candidates we can later interview and observe to complete the use-case modelling effort. Actors also determine the completeness of the system requirements, as they can be used to verify and validate the use cases upon completion.

Actors should be named with a noun or noun phrase.

When looking for actors, ask the following questions:

- Who or what provides input to the system?
- Who or what receives output from the system?
- Are interfaces required for other systems?
- Are there any events that are automatically triggered at a predetermined time?
- Who will maintain information in the system?

Identify Business Requirements Use Cases

Business Requirements capture user interactions in a technology- and implementation-free manner, showing only the most important use cases. When looking for the important use cases, ask the following questions:

- What are the main tasks of the actor?
- What information does the actor need from the system?
- What information does the actor provide to the system?
- Does the system need to inform the actor of any changes or events that have occurred?
- Does the actor need to inform the system of any changes or events that have occurred?

Construct Use-Case Model Diagram

Once the use cases and actors have been identified, a use-case model diagram can be used to graphically depict the system scope and boundaries. The processes of the use-case are outlined, and the relationships to actors are represented as lines.

Example 1: Create a set of use cases for the process of buying glasses from the viewpoint of the patient. The first step is to see an eye doctor who will give you a prescription. Once you have a prescription, you go to a glasses store, select your frames, and place your order. Once the glasses are made, you return to the store for a fitting and pay for them.

Solution

Note: the question says from the “view of the patient”. That is, you are expected to consider only patient interaction with the system

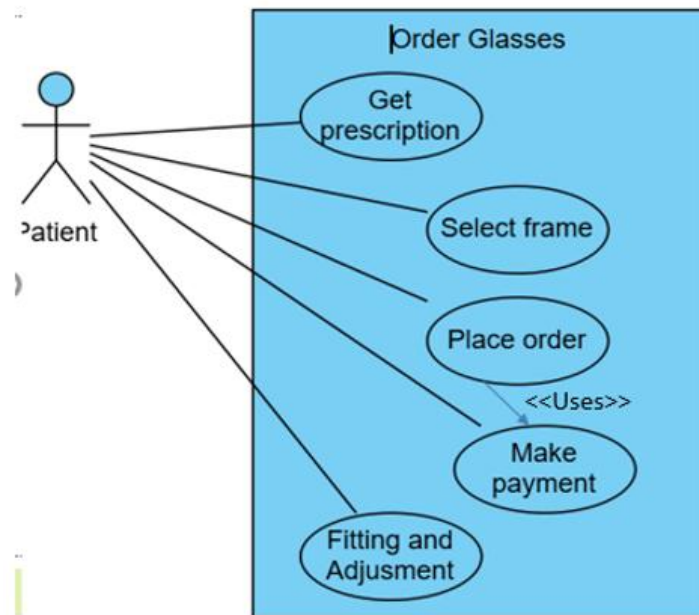


Figure 7: Example 1 Solution (Order Glasses Use case)

Example 2: Use case descriptions for the Get prescription, select frame, and place order use cases.

Solution

Use Case Name	Get Prescription
Actor	Patient
Description	Allows the patient to obtain an eye prescription
Preconditions	Patient is registered or visits the clinic
Main Flow	1. Patient requests eye test 2. System schedules/records test 3. Eye test is conducted 4. Prescription is generated 5. Prescription is provided to the patient
Alternative Flow	Test unavailable, reschedule appointment
Postconditions	A valid prescription is issued

Use Case Name	Select Frame
Actor	Patient
Description	Allows the patient to choose a suitable eyeglass frame
Preconditions	Patient has a prescription
Main Flow	1. Patient browses available frames 2. Patient selects preferred frame 3. System records selection
Alternative Flow	Frame not available, choose another frame
Postconditions	Frame choice is recorded

Use Case Name	Place Order
Actor	Patient
Description	Allows the patient to place an order for eye-glasses
Preconditions	Valid prescription; Frame selected
Main Flow	1. Patient confirms prescription and frame 2. System calculates cost 3. Patient confirms order 4. System creates order
Alternative Flow	Order cancelled by the patient
Postconditions	Order is successfully created

In-Text Questions (ITQs)

Create a description for the make payment use case.

In-Text Answers (ITAs)

Use Case Name	Make Payment
Actor	Patient
Description	Allows the patient to pay for the glasses
Preconditions	The order has been placed
Main Flow	1. System displays payment options 2. Patient selects payment method 3. Patient enters payment details 4. System processes payment 5. Payment confirmation is generated
Alternative Flow	Payment fails, retry or cancel
Postconditions	Payment is completed and recorded

2.2.3 Conclusion

Effective requirements modelling improves communication between technical and non-technical stakeholders. It supports requirements validation and provides a good foundation for system design and development. It also supports the testing process as it helps to check the developed system against requirements. By clearly defining system requirements through models such as use case diagrams and narratives, organisations reduce development risks, control scope, and increase the likelihood of delivering systems that meet business objectives and user expectations.

Summary

This section discusses requirements modelling using use cases. The elements of the use case diagram were explained, and the contents of the use case narrative and the process for requirement modelling were discussed.

Self-Assessment Questions

1. In a school grading system, a teacher can record and update grades. Recording grades involves saving them, while updating grades involves loading and saving them. The system administrator can generate report cards; however, these are distributed by a teacher. A teacher, an administrator, and a student can view grades by logging in to the system. Sketch a Use Case diagram to depict this scenario. Hint: Use the relationship.
2. Create a use case narrative for teacher record grades, teacher upgrade grade, and student view grades

Unit Activities

Read the content of Unit 1 in the study guide. Consult the recommended texts and research further online. (90 minutes)

Complete the self-introductory forum. Original post by Thursday. (10 minutes).
Complete replies to 2 of your classmates by Sunday at 11:59 p.m. (10 minutes)

Tutor-Marked Assignment

Complete the following and submit for grading. (60 minutes)

A sports centre offers facilities such as football pitches, tennis courts, basketball courts, and a gym. Customers can book these facilities for specific dates and time slots using an online booking system.

A Customer must first create an account and log in to the system. After logging in, the customer can view available sports facilities, check time slot availability, and make a booking for a preferred facility.

When making a booking, the system displays available time slots. The customer selects a suitable date and time, confirms the booking, and proceeds to make payment. Once payment is successful, the booking is confirmed and recorded in the system. The customer can also view, modify, or cancel bookings. The system sends a booking confirmation notification (via email or SMS) after a successful reservation.

A Staff/Admin manages the system by adding or updating facilities, setting availability schedules, and monitoring bookings. The admin can also generate reports on facility usage and customer bookings.

- i. Create a use case diagram for the sports centre booking system in this scenario.
- ii. Develop a use case narrative for ANY four use cases in the diagram

References/Further Readings

System Analysis and Design, Alan Dennis, Barbara H. Wixom, and Robert M. Roth, John Wiley & Sons, 5th Edition, 2012.

System Analysis and Design Methods, Jeffery L. Whitten and Lonnie D. Bentley, 7th Edition, McGraw-Hill- Irwin, 2007

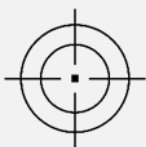
Object-Oriented Systems Analysis and Design Using UML, Simon Bennett, Steve McRob and Ray farmer, McGraw-Hill, Second Edition, 2002.

Software Engineering, Ian Sommerville (2004)7th Edition. Chapter 6.

Unit 3 — Process Modelling

INTRODUCTION

This unit discusses how the requirements definition can be clarified through a process model. Process modelling is important in the analysis and design of software systems. It helps analysts and developers understand user requirements, identify system functionality, data flow, and data store, and ensure that the final system aligns with business needs. A process model is a graphical representation of how a business system should operate. It illustrates the processes and activities performed and how data move among them. A process model can be used to document the current system (i.e., as-is system) or the new system being developed (i.e., to-be system) (Dennis, Wixon & Roth, 2012). Modelling a process “as-is” helps organisations to identify inefficiencies, delays, or redundancies and can then design an improved version (the “to-be” model) that enhances performance, reduces cost, and increases productivity.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Define process modelling and explain its benefits
- Explain the elements of a Data Flow Diagram
- Understand the basic concepts and constructs of a process model using Data Flow Diagram (DFD)
- Identify common errors and illegal data flows to be avoided in the DFD Diagram

2.3.1 Definition

A business process is a collection of activities designed to produce a specific output for a particular customer or market. It implies a strong emphasis on how the work is done within an organisation, in contrast to a product, which focuses on what.

Example 1: If you want to make a sandwich for lunch, you must carry out some activities such as bringing out the ingredients like bread, peanut, and butter from the cupboard, and bringing out the cream from the refrigerator. The next activity will be to mix the ingredients, then build the sandwich, and lastly package the lunch. This business process can be illustrated as shown in Figure 1.

The business goal (output) is the "Lunch". It consists of several activities called "processes".

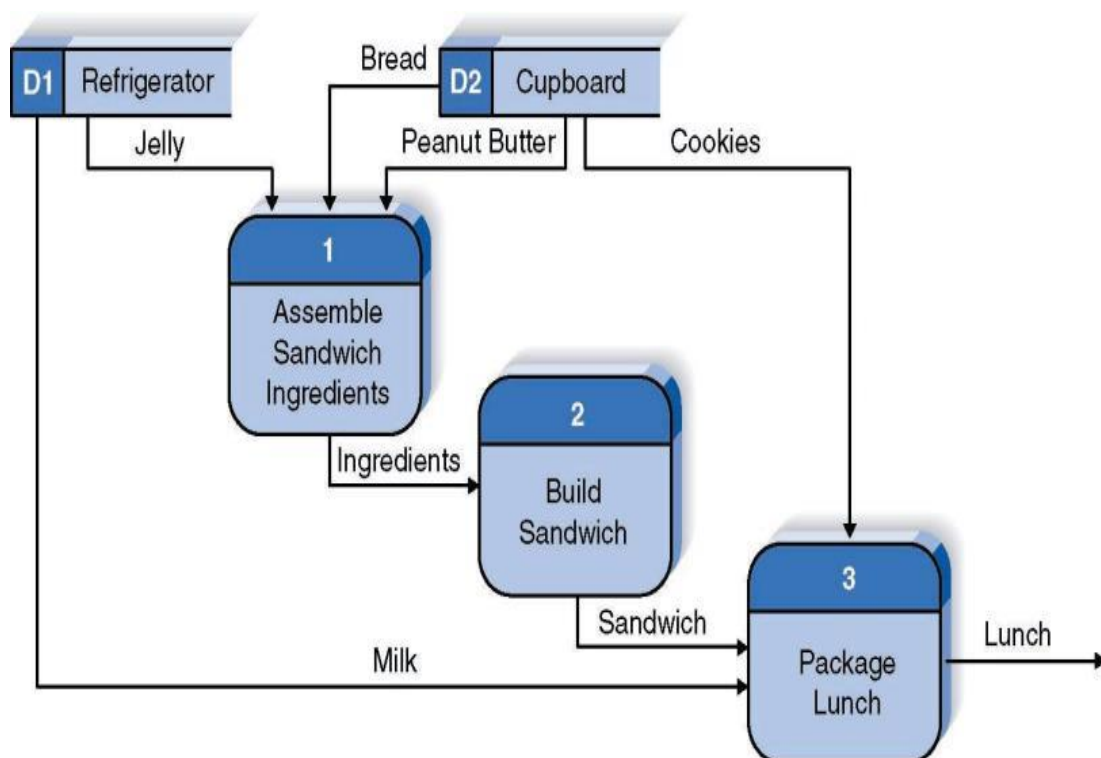


Figure 1: Simple process for making lunch

In-Text Questions (ITQs)

Name the processes involved in making lunch in example 1

In-Text Answers (ITAs)

1. Assemble sandwich ingredients
2. Build a sandwich
3. Package lunch

A process is a set of related activities or tasks performed in a sequence to achieve a specific goal or produce a desired output. It involves inputs, actions (steps), and outputs. Also, process a clear objective (e.g., assemble sandwich ingredients) and may involve people, systems, or machines.

A Process thus a specific ordering of work activities across time and place, with a beginning, an end, and clearly defined inputs and outputs: a structure for action.

A model is a pictorial representation of reality.

Process modelling is a technique for organising and documenting a system's processes, inputs, outputs and data stores. It is a formal and graphical way of representing how a business operates. The main tool of process modelling is the Data Flow Diagram (DFD).

- For example, processing a customer's order from an online store may involve:
- The customer placing an order
- Send IT-based information to the warehouse
- Sending IT-based information to the accounts department
- Generation of an invoice
- Stock picking
- Packing and recording
- Sending the appropriate IT-based information to the distribution hub
- Allocate and organise shipments for the vehicle drivers
- Delivery of the item

2.3.2 Importance of Process Modelling

- It allows designers and users to focus on the business aspects of systems;
- Process models help developers understand user requirements, design system functionality, and ensure the system meets business needs;
- It is cheap and easy to modify, especially with the assistance of Computer-Aided Software Engineering (CASE) tools;
- The non-technical notation is easily understood by business people, and

- DFDs allow isolation of parts of the system at any chosen level of detail.
- Process models act as formal documentation of how a system or business operates.

2.3.3 Data Flow Diagram (DFD)

The tool for process modelling is a Data Flow Diagram (DFD). A Data Flow Diagram shows business processes and the data that flows between them. A DFD depicts the flow of data through a system and the work or processing performed by that system. Thus, a Data Flow Diagram is a graphical representation of how data moves through a system. DFD shows the source of data input, the processes that act on the data, and the data store, without focusing on technical details such as programming. DFDs allow the analyst to determine:

- What does the organisation do?
- How does the organisation do it?

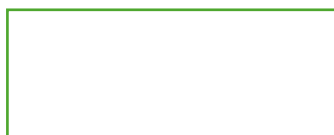
NOTE:

- DFDs are purely logical, not physical models.
- DFDs contain no implementation details.
- DFD is drawn first because it allows designers and users to concentrate purely on capturing business aspects.
- DFD is not contaminated by distractions of technical details.

DFD Elements

There are four DFD elements (symbols).

1. **External Agent/Entity:** An external entity is a person, organisation, or system outside the system boundary that interacts with the system. It is the source and destination of data.
 - It is depicted as a rectangle.
 - Nouns are usually used to name external agents. Examples: Student, Customer, Client, employee, organisation unit, another information system, etc.
 - External agent does not perform processing.
 - Named with a descriptive, singular noun



External Agent or Entity Symbol

2. **Data Flows:** They are directed arrows. They can also be described as pipes. Data flows in packets through the pipes. One flow arrow represents the movement of multiple data items. Data flow names should connote the actual information, e.g., Order Details, Receipt. It is named using nouns.



Data Flow Symbol

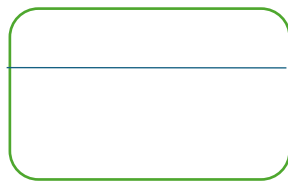
3. **Process:** it is depicted by a rectangle with round edges. A process represents an activity or operation that transforms input data into output data. It shows how the system processes the data. Data flows into the process, and the "process" works on the data flowing into it. The data flowing out has more value after it has been processed.

A process is always active. Examples: Create Student Record, Calculate Purchase Cost, Register Client.

Process name must contain verbs and must not be procedural.

Example: Add 1 to the counter ----- Procedural

Increment counter-----Process



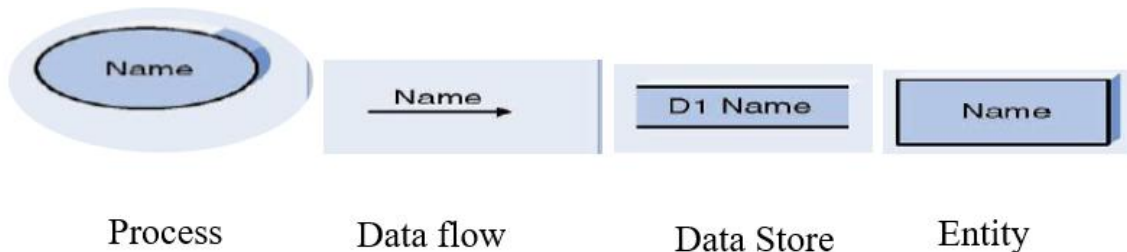
Process Symbol

Hints to identify logical process:

- It performs computations. E.g., calculate grade point average)
 - It makes decisions. E.g.: determine availability of ordered products)
 - It sorts, filters, or otherwise summarises data. E.g.: identify overdue invoices)
 - It organises data into useful information E.g.: generate a report or answer a question)
 - It triggers other processes E.g.: turn on the furnace or instruct a robot)
 - It uses stored data (create, read, update or delete a record)
4. **Data Store:** a place where data is stored for later use. The data never changes in any way in the data store, except that the data is used as input to a process, and the new (updated) data is returned to the data store. In naming data stores, avoid verbs and physical descriptions of how the data is stored. Examples: Student Master, Client List. Frequently implemented as a file or database. A data store is "data at rest" compared to a data flow that is "data in motion." Data store is represented as an open-ended rectangle (two parallel lines)



Note: this notation of the DFD Elements is the Gane-Sarson notation. Another popular notion used is the DeMarco and Yourdon symbol, described as follows:



Decomposition of DFDs

It is the iterative act of going from one single system to many component processes. That is, the process of **breaking down a complex system into smaller, more detailed sub-processes**. It helps analysts move from a **high-level overview** of a system to a **more detailed understanding** of how each part works. Each level of abstraction reveals more or fewer details.

Levels of DFD Decomposition

1. **Context Diagram:** Represents the entire system as one single process and shows the data flows to and from external entities. Data stores are usually not included in the context diagram unless they are "owned" by systems or processes other than the one being documented.

It shows only:

- External entities
 - Data flows in and out of the system
 - Example: ATM System as one process interacting with the Customer
2. **Level 0 DFD:** Breaks the main system into major sub-processes. A DFD that represents a system's major processes, data flows and data stores at a higher level. Shows all the processes that comprise the overall system. Shows how information moves from and to each process. It adds data stores. All process models have exactly one level 0 DFD.

Example: ATM System level 0 DFD will show:

- Key processes
- Data stores
- Data flows

Example:

- Authenticate User,
- Withdraw Money,
- Deposit Money
- Check Balance, etc

3. Level 1 (and Lower Levels)

Further breaks down each process into more detailed sub-processes. It continues until the process is simple enough to understand

For example, the "Withdraw Money" process is broken into:

- Validate PIN process
- Check Balance process
- Dispense Cash process

When decomposing a DFD, you must conserve inputs and outputs from processes at the higher level of decomposition. This is called Balancing. It ensures consistency across all levels

Example 1: A bank provides an Automated Teller Machine (ATM) system that allows customers to perform basic banking transactions such as withdrawing cash, checking account balance, and printing receipts. Create a Process 0, level 0 and Level 1 DFD for the user's interaction with the system.

Solution

Process 0 (Context Diagram): The context diagram represents the entire system as a single process. The entities, the input to the system and the output. Based on the scenario, we are only concerned about one entity: the customer.

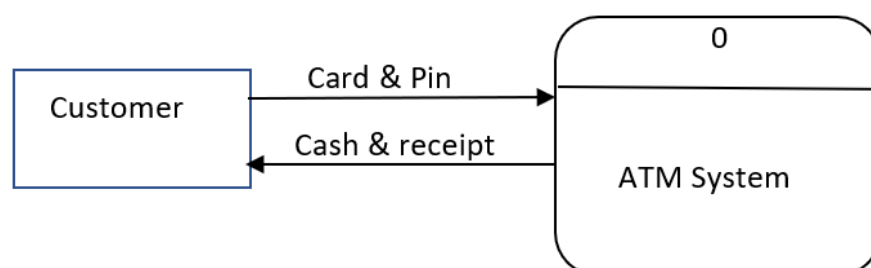


Figure 1: ATM System Context Diagram

Level 0: Shows the major processes, data flow and data store. In this example, three of the major processes that can take place in an ATM are captured as shown in Figure 2

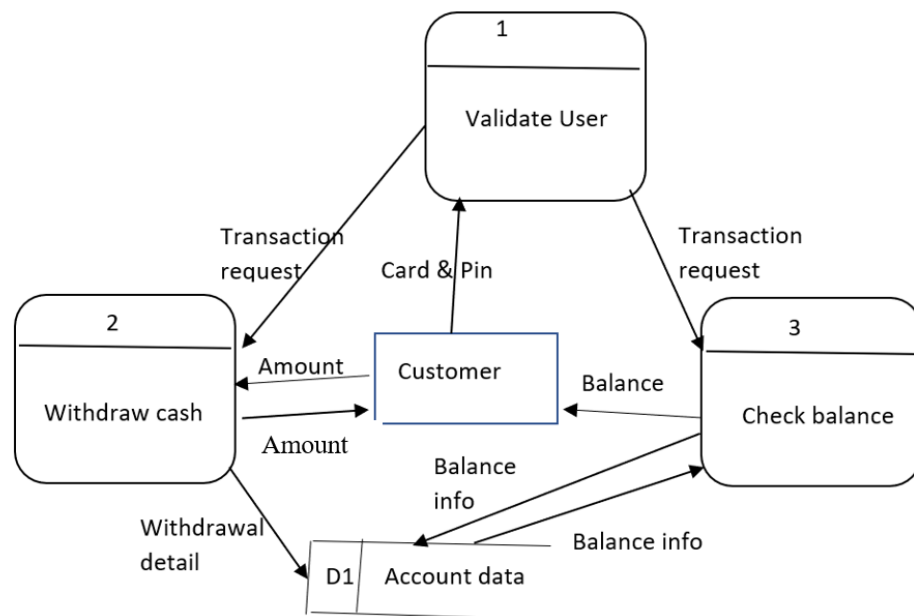


Figure 2: Level 0 DFD for ATM System

Level 1: This level further breaks a process into smaller processes. Withdrawal cash can be further broken into its constituent processes, as shown in Figure 3

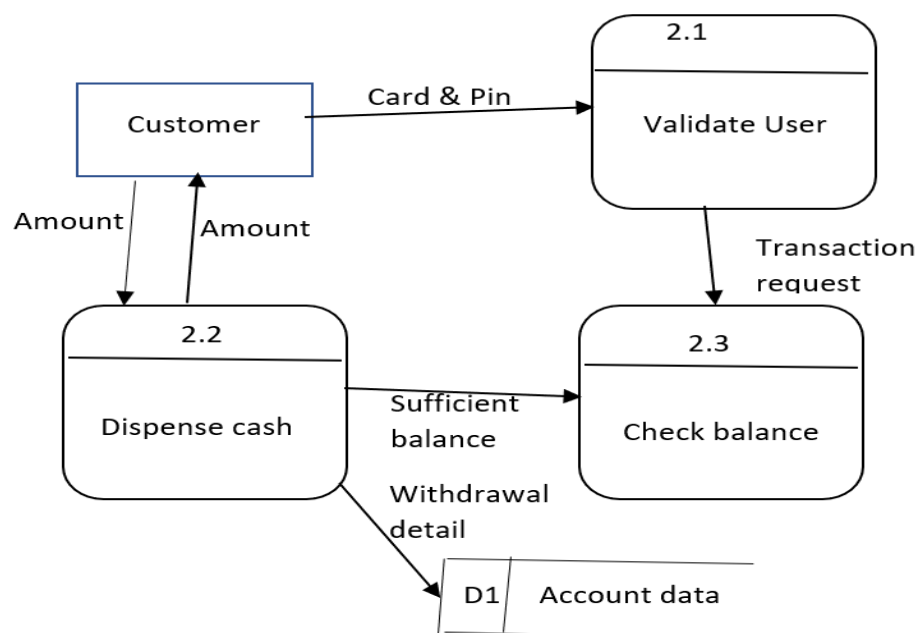


Figure 3: Level 1 DFD for ATM System

NOTE: There can be only one level 0 DFD in a system. Level 1 and other lower levels can be many based on the number of processes in the higher level that can be broken down.

Physical data stores, such as printed receipts and cash cassettes in the ATM machine, can only be represented as data stores in a conceptual sense because DFD does not represent physical data stores.

In-Text Questions (ITQs)

What is balancing in DFD?

In-Text Answers (ITAs)

Balancing is conserving inputs and outputs from processes at a higher level at the next level of decomposition of DFD.

Common Mechanical Errors in DFD

When developing system models using DFD, three (3) common errors need to be avoided. These also help system analysts to identify the correctness of the model that is being produced.

The errors are:

1. **Black hole:** A black hole is when a process has inputs but no outputs. Data enters the process and then disappears.
2. **Miracle:** A miracle is when a process has outputs but no input.
3. **Gray hole:** A gray hole is when the inputs of a process are insufficient to produce the output

In Figure 4, Process 3.12 has inputs but no outputs. This is a black hole because data enters the process and then disappears. In most cases, the modeller simply forgot the output.

Process 3.1.2 has outputs but no input. This is a miracle! In this case, the input flows were likely forgotten.

Process 3: The inputs are insufficient to produce the output. This is a gray hole. There are several possible causes, including:

- a. a misnamed process,
- b. misnamed inputs and/or outputs,
- c. incomplete facts.

Gray holes are the most common errors – and the most embarrassing. Once handed to a programmer, the input data that flows to a process (to be implemented as a program) must be sufficient to produce the output data that flows.

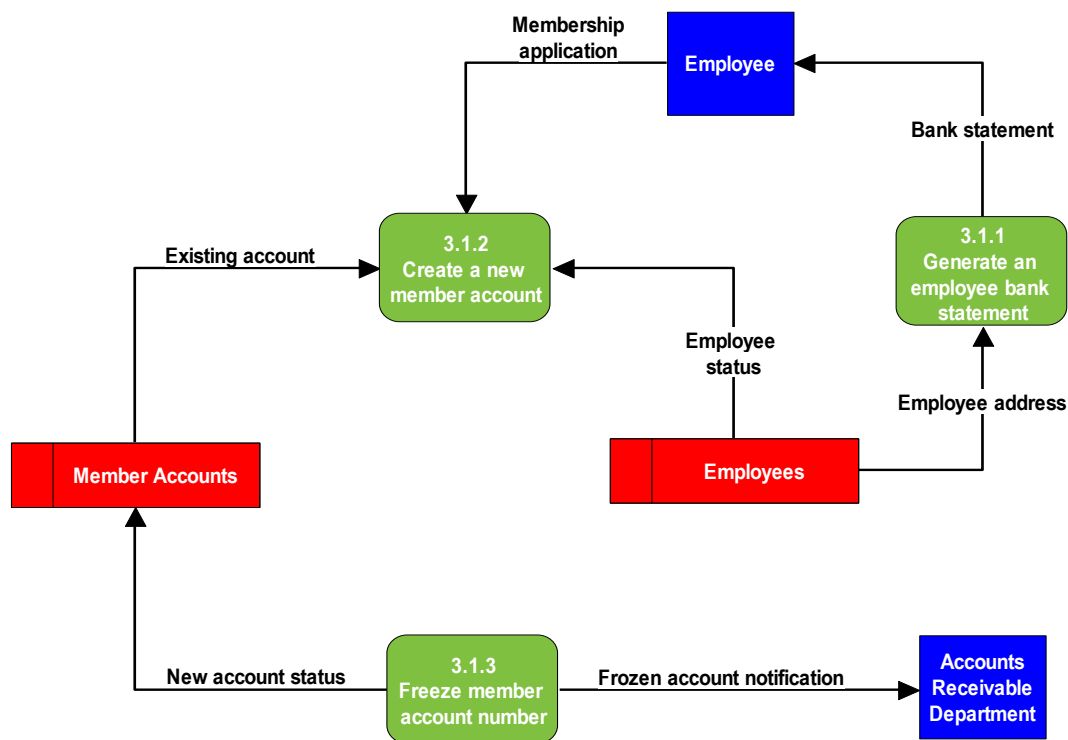


Figure 4: Common Errors in DFD

Illegal Data Flows

As shown in Figure 5, all of the data flows on the left side of the. The corrected diagrams are shown on the right side.

- i. Data should not flow from an external agent to another external agent because DFDs do not model physical interaction between entities. Rather, the data must flow through a process
- ii. Data cannot flow from a data store directly to an external agent. Rather, the external agent needs to interact with a process to access data in the data store.
- iii. Data cannot flow from one data store to another directly but must pass through a process.
 - Data Flow Names: Should discourage premature commitment to any possible implementation.
 - Should be descriptive nouns and noun phrases that are singular, as opposed to plural.
 - Should be unique.
 - Use adjectives and adverbs to help to describe how processing has changed a data flow.
 - No data flow should ever go completely unnamed.

- Data flow names should describe the data flow without describing how the flow is or could be implemented.
- All data flows must begin or end at a process, because data flows are the inputs and outputs of a process.

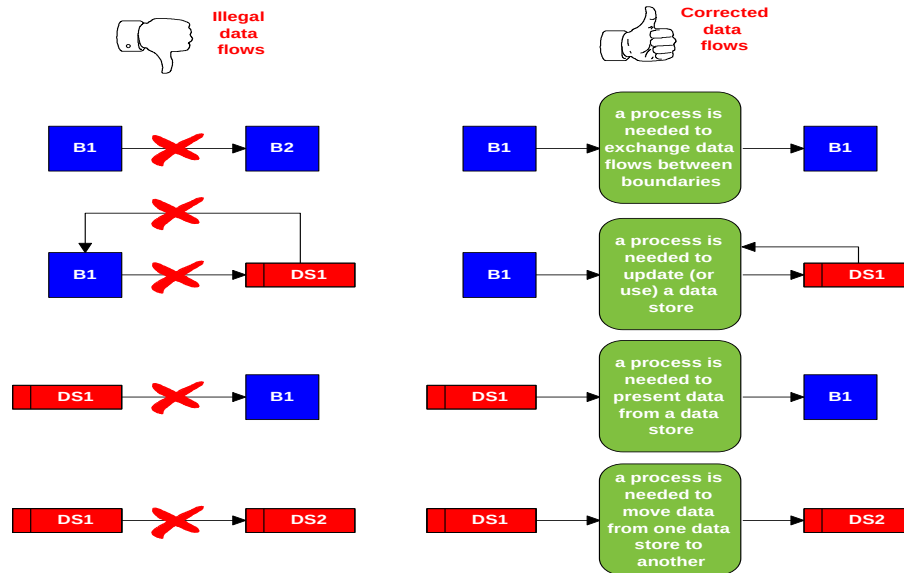


Figure 5: Illustration of Illegal Data Flow and Corresponding Correct Data Flow

2.3.4 Conclusion

DFD is a process model. Logical process models describe processes without suggesting how they are conducted. Logical models show what a system 'is' or 'does', depicting the system independent of any technical implementation. They reduce the risk of missing business requirements because we are too preoccupied with technical results. Logical models allow us to communicate with end-users in non-technical or less technical languages.

Iteration is essential to creating an effective DFD. Even skilled analysts rarely produce a perfect diagram on their first attempt. Usually, an initial version is drawn to understand the arrangement of processes, data flows, data stores, and external entities. A revised version is then created to improve clarity and minimise crossing data flows. In practice, a DFD is refined multiple times before reaching its final form.

After the Level 0 DFD that identifies all the major processes in the system, create lower-level DFDs for each process in the Level 0 DFD that needs a Level 1 DFD. This is repeated for level 2 DFD until there is no process that can be broken into smaller processes in the system

Summary

The unit introduced the concept of modelling user requirements with a process model using a Data Flow Diagram (DFD). The elements of DFD were explained, and the process of decomposing a business process was illustrated with an example. Three common mistakes to avoid when drawing a DFD diagram are black hole, miracle, and gray hole. Illegal data flows, such as data flowing from an entity to a data store, must be avoided.

Self-Assessment Questions

1. What do you understand by Function Decomposition? Relate it to DFD.
2. Given that you are to develop a food ordering system for Covenant University Cafeteria 2, draw the process 0 and level 0 DFD of the proposed system

Unit Activities

1. Read the content of Unit 1 in the study guide. Consult the recommended texts and research further online. (90 minutes)
2. Complete the self-introductory forum. Original post by Thursday. (10 minutes). Complete replies to 2 of your classmates by Sunday at 11:59 p.m. (10 minutes)

Tutor-Marked Assignment

1. Complete the following and submit for grading. (60 minutes)

References/Further Readings

- System Analysis and Design, Alan Dennis, Barbara H. Wixom, and Robert M. Roth, John Wiley & Sons, 5th Edition, 2012.
- System Analysis and Design Methods, Jeffery L. Whitten and Lonnie D. Bentley, 7th Edition, McGraw-Hill- Irwin, 2007
- Object-Oriented Systems Analysis and Design Using UML, Simon Bennett, Steve McRob and Ray farmer, McGraw-Hill, Second Edition, 2002.
- Software Engineering, Ian Sommerville (2004)7th Edition. Chapter 6.

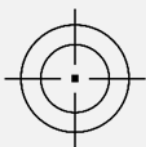
Unit 4 — Data Modelling and Entity-Relationship Diagramming

INTRODUCTION

Think about registering for courses on your school's portal. Behind that simple screen, the system is quietly keeping track of two things: you (a Student) and the courses you pick (a Course), plus the link between them showing exactly which courses you registered for. Deciding what pieces of information to keep, and how they connect, is exactly what this unit is about.

The previous unit looked at process modelling, which shows what a system does and how data moves through it using a Data Flow Diagram (DFD). This unit looks at the data itself. Data modelling is the analysis of data objects and the relationships between them. It is usually the first step in database design, and it produces a diagram called an Entity-Relationship Diagram (ERD).

While a DFD answers the question "what does the system do with data?", an ERD answers the question "what data does the system need to keep, and how is that data connected?" The two models complement each other: the DFD shows the verbs of the system, and the ERD shows the nouns. Together they form the full requirements picture needed before a database can be built.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Define data modelling and explain its purpose
- Identify the basic elements of an Entity-Relationship Diagram
- Explain the three types of relationship cardinality
- Read and interpret an ER diagram
- Construct a simple ER diagram from a given scenario

2.4.1 What is Data Modelling?

Data modelling is the process of identifying the data objects (entities) an organisation needs to keep track of, the properties of each object, and how the objects relate to one another. A data model is a diagram that shows these relationships in a way that is independent of any database technology, so the same model could later be implemented in Microsoft Access, MySQL, or any other database system without changing the underlying design. It gives analysts, designers, and users a shared picture of the information the system must manage, long before anyone writes a line of database code.

Benefits of data modelling include:

- It clarifies exactly what data the organisation needs to store, reducing the risk of missing or duplicated information
- It provides a stable foundation for physical database design, since tables, columns, and keys can be derived directly from the model
- It improves communication between analysts, designers, and business users, since the diagram uses simple, non-technical symbols that anyone can read after a short explanation
- It exposes rules and constraints in the business, such as which relationships are mandatory and which are optional, before those rules are buried inside program code

2.4.2 Elements of an Entity-Relationship Diagram

An ERD is built from four main elements: entities, attributes, identifiers, and relationships. Each element has a specific symbol, and once you can recognise the four symbols, you can read almost any ER diagram.

Entity

An entity is a person, place, object, event, or concept about which the organisation keeps data. It is a real-world thing that is distinguishable from other things, and it always has more than one instance; a single Toyin Ojo is not an entity, but Student (the category Toyin Ojo belongs to) is. An entity is drawn as a rectangle and named with a singular noun, such as Student rather than Students, since the rectangle represents the whole category, not any specific member of it.

Attribute

An attribute is a piece of information captured about an entity; it is a property of that entity. Attributes of a Student entity might include student_name, student_level, and student_programme. Only attributes that the organisation needs should be included in the model, since every extra attribute adds a column that must later be stored, validated, and maintained. An attribute is drawn as an oval connected to its entity. Some attributes are composite, meaning they can be broken down into smaller attributes of their own; for example, an Address attribute may be composed of Street, City, and State, and the model can show these as smaller ovals branching off the Address oval.

Identifier

The identifier is the attribute, or combination of attributes, that uniquely identifies each instance of an entity. It is also called the primary key. For a Student entity, the identifier would typically be student_id, since two students could share the same name but never the same ID. Choosing a good identifier matters: it should never be blank, and it should never need to change once assigned, which is why systems generally prefer an assigned ID number over something like a phone number or email address.

Relationship

A relationship is an association between two or more entities. It is drawn as a line connecting the entities involved, and is usually named with an active verb, such as Registers, Teaches, or Supervises, so that the diagram can almost be read like a sentence: Student Registers for Course. A relationship line carries three properties that together form the relationship statement:

- The link phrase, which is the verb describing the relationship
- The degree or cardinality, which states how many instances of one entity relate to instances of the other
- The participation or ordinality, which states whether the relationship is mandatory or optional

Figure 4.1 below shows all four symbols side by side, using the Student entity as a running example, before they are combined into a complete diagram later in this unit.

ER Diagram Notation: The Four Basic Symbols

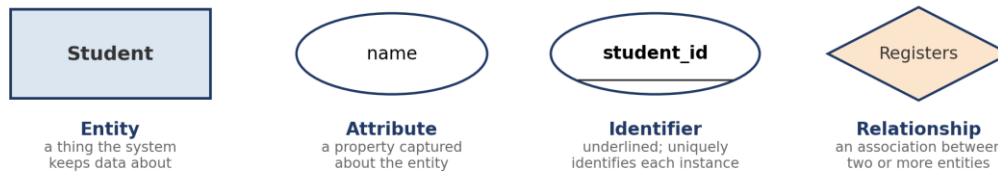


Figure 4.1: The four basic ERD symbols

2.4.3 Cardinality of Relationships

Cardinality specifies how many instances of one entity can relate to a single instance of another entity. There are three common cardinal relationships, and recognising which one applies is usually the trickiest part of building an ERD:

- One-to-one (1:1): each instance of Entity A relates to exactly one instance of Entity B, and vice versa. For example, each employee has one login account, and each login account belongs to one employee.
- One-to-many (1:M): one instance of Entity A relates to many instances of Entity B, but each instance of B relates back to only one instance of A. For example, one customer can place many orders, but each order belongs to one customer only.
- Many-to-many (M:N): instances of Entity A can relate to many instances of Entity B, and instances of B can relate to many instances of A. For example, a student can register for many courses, and a course can have many students. A many-to-many relationship is normally resolved by introducing an associative entity, such as Enrolment or Registration, when the ERD is converted into database tables.

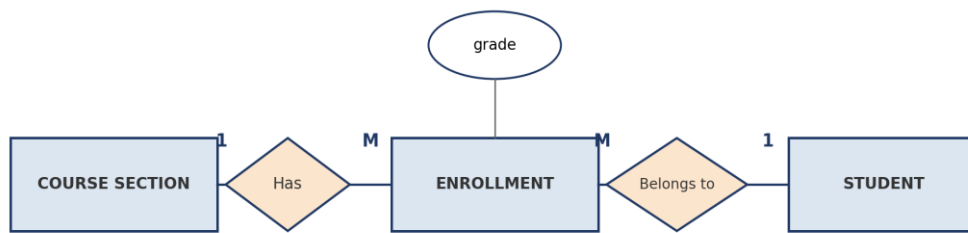
Ordinality, sometimes shown alongside cardinality, states whether the minimum number of instances required is zero (optional) or one (mandatory). For example, a Student may be required to have at least one Enrolment (mandatory), while a Course might exist in the catalogue even before any student has registered for it (optional).

Reading an ER Diagram

An ERD is read one relationship line at a time, in both directions. Consider three entities: Course Section, Enrolment, and Student, connected by the relationships Has

and Belongs to. Reading from left to right: one Course Section has many Enrolments, each with its own grade, and each Enrolment applies to one specific Student. Reading the same diagram from right to left: one Student has many Enrolments, each with its own grade, and each Enrolment applies to one specific Course Section. A database built from this model could then produce grade lists showing every student and their grade in each course section, as well as transcripts showing every grade a given student has earned.

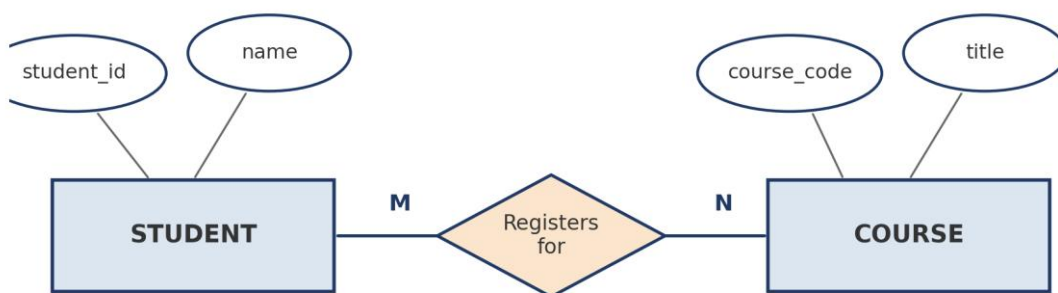
Reading an ER Diagram: Course Section, Enrollment, and Student



*Left to right: one COURSE SECTION has many ENROLLMENTS.
 Right to left: one STUDENT has many ENROLLMENTS. Each ENROLLMENT links exactly one STUDENT to exactly one COURSE SECTION, and carries its own grade.*

Figure 4.2: The Course Section, Enrolment, and Student example, read in both directions

Sample Entity-Relationship Diagram



*One STUDENT may register for many COURSEs (M);
 one COURSE may have many STUDENTs registered (N).*

Figure 4.3: A simple ER diagram showing the Student-Course relationship

2.4.4 Constructing an ER Diagram: A Worked Example

Scenario: A university wants to keep track of students and the courses they register for. Each student can register for many courses, and each course can have many students registered. The steps below walk through building the diagram shown in Figure 4.3.

- Identify the entities. Here, the two entities are Student and Course.
- Identify the attributes of each entity, and mark the identifier. Student has student_id (identifier) and name. Course has course_code (identifier) and title.
- Identify the relationship between the entities and name it with a verb. Here, the relationship is Registers for.
- Determine the cardinality of the relationship. One student can register for many courses (M), and one course can have many students (N), so this is a many-to-many relationship.
- Draw the diagram: place the two entities as rectangles, connect them through a relationship diamond labelled with the verb, mark the cardinality (M and N) on either side of the diamond, and attach the attribute ovals to their entities.

The finished diagram, following exactly these five steps, is what is shown in Figure 4.3 above: two entity rectangles, a relationship diamond between them carrying the cardinality labels M and N, and the attribute ovals branching off each entity.

Summary

This unit introduced data modelling as the technique used to capture what data a system must store and how that data is related. The main elements of an Entity-Relationship Diagram are entities, attributes, identifiers, and relationships. Relationships are described by cardinality, which may be one-to-one, one-to-many, or many-to-many, and by ordinality, which states whether participation is mandatory or optional. Reading and constructing simple ER diagrams prepares the ground for later database design work.

Self-Assessment Questions

1. Distinguish between an entity and an attribute, giving one example of each.

2. Explain, with an example, the difference between a one-to-many and a many-to-many relationship.
3. A hospital keeps records of Doctors and Patients. A doctor can attend to many patients, and a patient may be attended to by many doctors over time. Draw an ER diagram for this scenario, including at least two attributes for each entity

Unit Activities

1. Read through the worked example in section 4.4 and redraw Figure 4.3 from memory, then check it against the original (30 minutes)
2. Pick any small business you are familiar with (for example, a bookshop or a pharmacy) and list the entities it would need to keep data on (20 minutes)

Tutor-Marked Assignment

1. A restaurant wants to keep track of Customers and the Orders they place. A customer can place many orders, but each order belongs to one customer. Each order also contains one or more Menu Items, and a menu item can appear on many orders.
 - a. Identify all the entities involved.
 - b. List at least three attributes for each entity and mark the identifier.
 - c. Draw the complete ER diagram, showing all relationships and their cardinality
2. A car rental company wants to keep track of Customers and the Vehicles they rent. A customer can rent many vehicles over time, and a vehicle can be rented by many different customers over time, but never by more than one customer at once.
 - a. Identify all the entities involved, including any associative entity needed to record each individual rental.
 - b. List at least three attributes for each entity and mark the identifier for each.
 - c. Draw the ER diagram, stating the cardinality and ordinality of every relationship shown

References/Further Readings

- Dennis, A., Wixom, B. H., & Roth, R. M. (2012). Systems Analysis and Design (5th ed.). John Wiley & Sons.
- Whitten, J. L., & Bentley, L. D. (2007). Systems Analysis and Design Methods (7th ed.). McGraw-Hill Irwin.

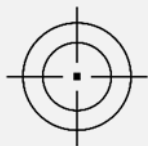
Module Three

Unit 1 — Data Modelling and Entity-Relationship Diagramming

INTRODUCTION

Open the Contacts app on your phone. Every contact you see, Mum, your landlord, your best friend, looks and behaves the same way: each has a name, a number, and a photo (data), and each can be called, messaged, or edited (behaviour). You did not design a new type of contact every time you saved one; you simply created a new instance of the same underlying idea of what a contact is. That underlying idea is what object-oriented design calls a class, and each saved contact is an object.

The earlier modules of this course used structured techniques: DFDs to model processes and ER diagrams to model data, as two separate views of the same system. Object-oriented design (OOD) takes a different approach. Instead of separating process and data, it combines them into a single unit called an object. This unit introduces the concept of object orientation and the notation used to describe object-oriented systems, the Unified Modelling Language (UML).



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Explain the concept of object orientation in software design
- Describe how a software design can be represented as a set of interacting objects
- Describe the activities involved in the object-oriented design process
- Explain the purpose of the Unified Modelling Language (UML) and its main diagram types

3.1.1 What is Object Orientation?

Object-oriented analysis and design are the process of modelling a software application using object-oriented techniques. Rather than modelling a system as a set of processes acting on separate data stores, object orientation models the system as a collection of objects that each manage their own data (state) and their own behaviour (operations). A running system, in this view, is simply a set of objects sending messages to one another, each responding according to its own class definition.

Objects and Classes

An object is a single instance of something in the system, such as a particular student or a particular order. Every object has:

- **State:** the data held by the object, represented as attributes, such as a specific student's ID and name
- **Behaviour:** the operations the object can perform, represented as methods or functions, such as registering for a course
- **Identity:** something that distinguishes it from every other object, even if its attributes are the same as another object's, in the same way two students could coincidentally share a name but never share a student ID

A class is the blueprint from which objects are created. It defines the attributes and operations that all objects of that type will have, but it does not itself hold any student's data; it only describes the shape that every Student object will follow. For example, the class Student defines that every student object has a name and a student ID, and can register for a course; a specific student such as Toyin Ojo is then an object, or instance, of the Student class. Objects interact with one another by sending messages, essentially calling one another's operations, which is how a system built this way actually gets work done.

A Class and Three of Its Objects

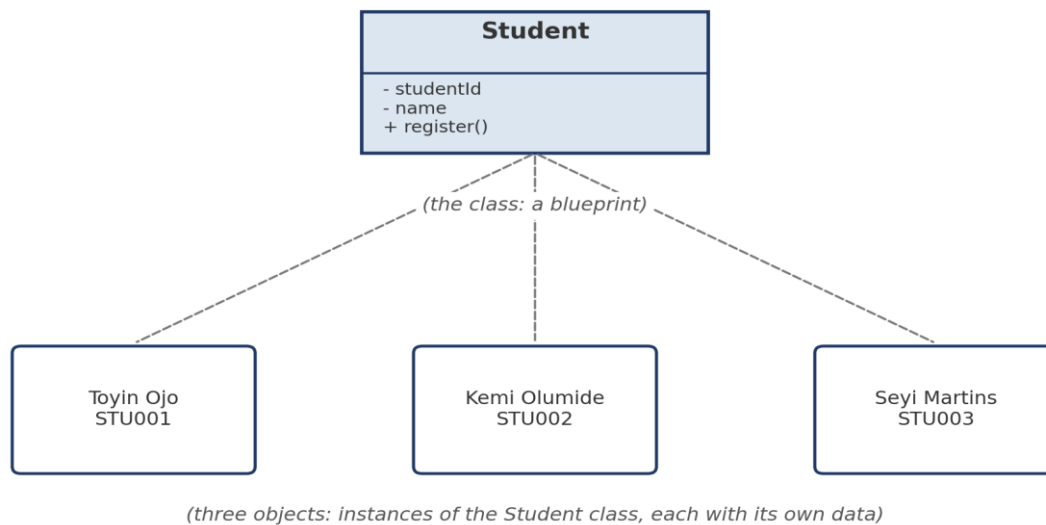


Figure 1.1: A class (the blueprint) and three of its objects (instances)

Core Object-Oriented Concepts

- **Encapsulation:** bundling an object's data and the operations that act on that data together and hiding the internal details from the outside. Other parts of the system interact with the object only through its defined operations, so, for example, other modules can call a Student object's `register()` operation without needing to know how that operation is implemented internally.
- **Inheritance:** allowing one class to be defined as a specialised version of another. For example, a Postgraduate Student class can inherit the attributes and operations of a general Student class, while adding its own specific attributes, such as a supervisor, which avoids repeating the shared attributes and operations in both classes.
- **Polymorphism:** allowing different classes to respond to the same operation call in ways appropriate to each class. For example, a `calculateFee()` operation might work differently for an Undergraduate Student object than for a Postgraduate Student object, even though both are called the same way, so the rest of the system does not need to know which specific type of student it is dealing with.

3.1.2 The Object-Oriented Design Process

Object-oriented design typically follows these broad activities:

1. Identify the candidate classes relevant to the system, usually by examining the requirements model built during analysis (including the use cases from Module Two, Unit 2)

2. Identify the relationships between classes, such as association, aggregation, composition, and inheritance
3. Identify the attributes and operations that belong to each class
4. Model the classes and their relationships in a class diagram
5. Model how objects interact to fulfil each use case, typically using a sequence diagram
6. Check that the classes identified are consistent with the use cases produced during requirements modelling

Unlike the strict, linear order suggested above, these activities are usually iterative: as class diagrams and sequence diagrams are built, gaps or errors in the requirements are often discovered and fed back into earlier steps. For example, building a sequence diagram for a Register for Course use case might reveal that a class needs an extra operation that nobody thought of while first listing the classes.

The Object-Oriented Design Process

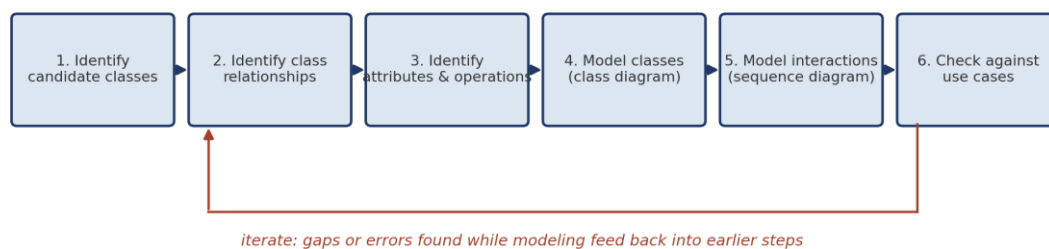


Figure 1.2: The object-oriented design process, showing the feedback loop

3.1.3 The Unified Modelling Language (UML)

The Unified Modelling Language (UML) is the industry-standard notation for documenting object-oriented analysis and design. UML is not a method in itself; it is a common visual language that different object-oriented methods can use to describe a system, in much the same way that standard road signs can be understood by drivers regardless of which driving school they attended. UML diagrams are broadly grouped into two categories:

- Structural diagrams, which describe the static structure of a system: the classes, objects, and physical components that make it up, and how they relate

to one another. Examples include class diagrams, component diagrams, and deployment diagrams.

- Behavioural diagrams, which describe how the system behaves over time: how objects interact, how control flows through the system, and how objects change state. Examples include use case diagrams, sequence diagrams, activity diagrams, state chart diagrams, and collaboration diagrams.

Figure 1.3 maps out where each diagram sits in this classification, and which unit covers it, before Units 2 and 3 introduce each one individually.

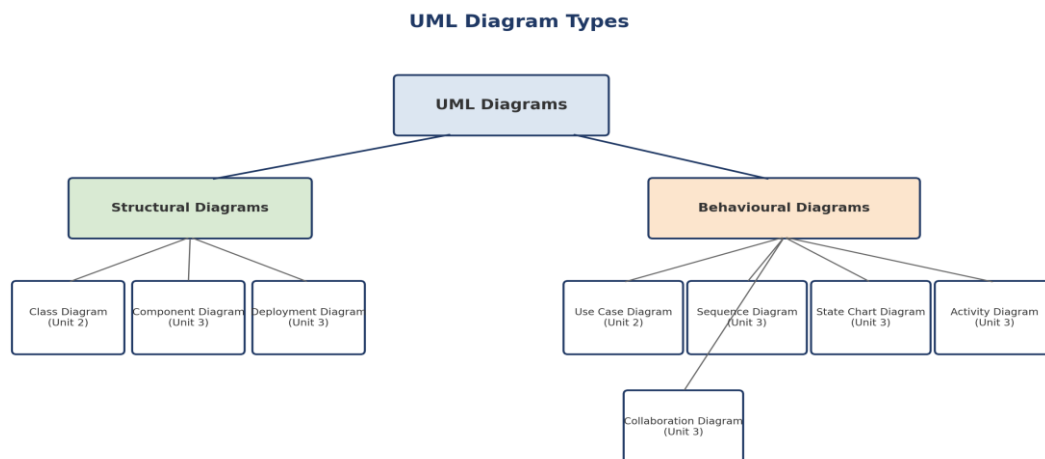


Figure 1.3: UML diagram types, grouped as structural or behavioural

The next two units look at these diagrams in more detail. Unit 2 covers structural and use-case modelling, and Unit 3 covers the remaining behavioural diagrams.

Summary

This unit introduced object orientation as an alternative to the structured, process-and-data-separated approach used earlier in this course. Objects combine state and behaviour, are created from classes, and are connected through encapsulation, inheritance, and polymorphism. The object-oriented design process moves from identifying candidate classes, through modelling their relationships and behaviour, to checking consistency with the use cases identified during analysis. UML is the standard notation used to document this process, made up of structural and behavioural diagrams.

Self-Assessment Questions

1. Explain, in your own words, the difference between an object and a class.
2. Give one example each of encapsulation, inheritance, and polymorphism, drawn from a system you are familiar with.
3. List the main activities in the object-oriented design process, in order.

Unit Activities

1. List five classes that might appear in a library management system, and briefly state one attribute and one operation for each (25 minutes)
2. Redraw Figure 1.1 for a different class of your choice (for example, Book or Product), showing the class and at least three of its objects (20 minutes)

Tutor-Marked Assignment

1. Consider a simple hotel booking system, where a Guest books a Room for a period.
 - a. Identify at least three candidate classes for this system.
 - b. For each class, list two attributes and two operations.
2. The same hotel offers both Standard Rooms and Suites, and both charge a booking fee, but the fee is calculated differently for each.
 - a. Explain, with reference to this scenario, how inheritance could be used to model Standard Room and Suite.
 - b. Explain how polymorphism would allow a single calculateFee() call to behave correctly, whether it is called on a Standard Room object or a Suite object.

References/Further Readings

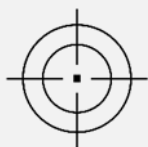
- O'Docherty, M. (2005). Object-Oriented Analysis and Design: Understanding System Development with UML 2.0. John Wiley & Sons.
- Bennett, S., McRobb, S., & Farmer, R. (2002). Object-Oriented Systems Analysis and Design Using UML (2nd ed.). McGraw-Hill.

Unit 2 — Structural and Use-Case Modelling in UML

INTRODUCTION

Think about your university's course registration app. If you looked at the class diagram behind it, you would probably find classes like Student, Course, and Enrolment, each with its own data and behaviour. If you instead looked at the use case diagram, you would see the actions you can perform, such as Register for Course or View Grades, and who is allowed to perform them. This unit shows how to build both views of the same system.

This unit looks at two of the most used UML diagrams: the class diagram, which shows the static structure of a system's classes, and the use case diagram, which shows how external users interact with the system. Together they form the core of object-oriented requirements modelling, and in practice they are usually developed side by side, each one checked against the other.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Describe the purpose and elements of a class diagram
- Identify the main types of relationship between classes
- Describe the purpose and elements of a use case diagram
- Construct a simple class diagram and use case diagram from a scenario.

3.2.1 The Class Diagram

A class diagram depicts the object structure of a system. It shows the classes in the system, their attributes and operations, and the relationships between them. A class is drawn as a rectangle divided into three compartments: the class name at the top, its attributes in the middle, and its operations at the bottom, so that everything you need to know about a class is visible in one shape.

Relationships Between Classes

- **Association:** a general relationship between two classes, showing that objects of one class are connected to objects of another. It is drawn as a line, often labelled with the multiplicity (how many objects of one class relate to how many of the other), such as 1 or M; for example, one Student is associated with many Enrolment records.
- **Aggregation:** a special form of association representing a whole-part relationship, where the part can exist independently of the whole. For example, a Department has Lecturers, but a lecturer can exist without belonging to that specific department, since they could transfer to another department and continue to exist as a Lecturer object.
- **Composition:** a stronger form of aggregation, where the part cannot exist without the whole. For example, an Order is made up of Order Lines; if the order is deleted, its order lines no longer make sense on their own and are deleted along with it.
- **Inheritance (generalisation):** a relationship where one class (the subclass) is a specialised version of another (the superclass), and inherits its attributes and operations. It is drawn as a line with a hollow triangle pointing to the superclass, and is the same relationship discussed in Module Three, Unit 1 in connection with a Postgraduate Student class specialising a general Student class.

These four relationships look similar briefly, a line is a line, but the small symbol at one end of the line is what tells them apart. Figure 2.1 places all four side by side so the notation can be compared directly.

Class Relationship Notations

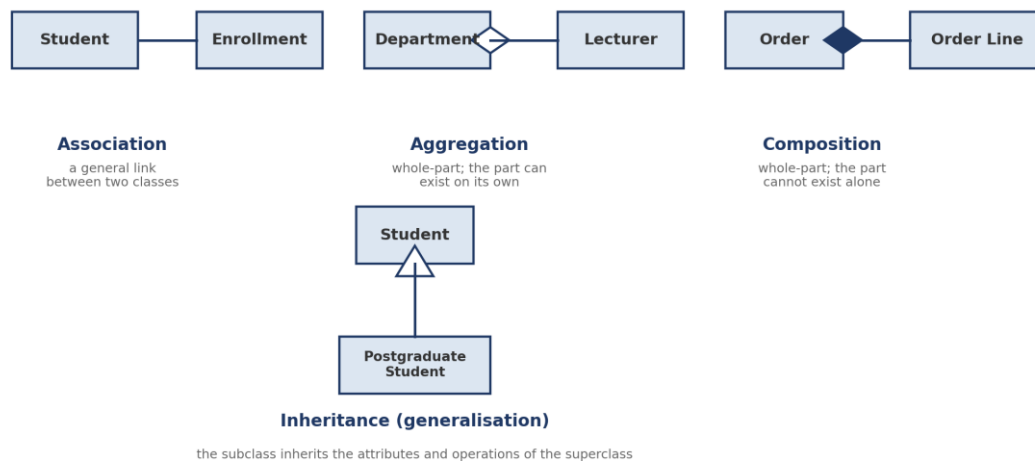


Figure 2.1: The four class relationship notations compared

Building a Class Diagram

The steps typically followed when building a class diagram are:

1. Identify the candidate classes from the requirements model, particularly the use case diagram and narratives
2. Identify the relationships and multiplicity between classes, such as association, aggregation, composition, and inheritance
3. Identify the attributes (properties) of each class
4. Identify the operations (behaviour) of each class
5. Check that the classes identified are consistent with the use cases already produced

Applying these steps to the Student, Enrolment, and Course classes produces the diagram in Figure 2.2: Student and Course are the two main classes identified in step 1, Enrolment is the associative class needed because a student can register for many courses and a course can have many students (step 2), and each class's attributes and operations were then added in steps 3 and 4.

Sample Class Diagram

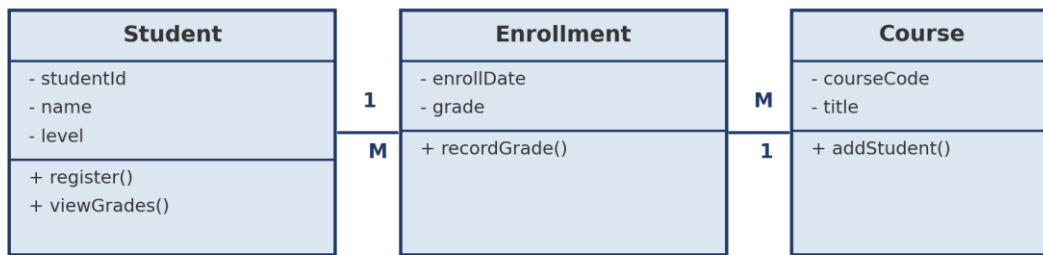


Figure 2.2: A sample class diagram showing Student, Enrolment, and Course

3.2.2 The Use Case Diagram

Although use case diagrams were introduced in Module Two, Unit 2 as part of requirements modelling, they are also considered a UML behavioural diagram, since they depict how external agents (users and other systems) interact with the system. A use case diagram shows actors, use cases, and the relationships between them, giving a high-level view of the functions the system must support, without describing how those functions are implemented internally.

Two relationships between use cases are commonly used beyond the basic association between an actor and a use case:

- **Include:** a base use case always requires the behaviour of another use case to complete. For example, Record Grades includes Save Grades, since every act of recording grades must end with saving them.
- **Extend:** a use case optionally adds extra behaviour to a base use case under certain conditions. For example, Place Order might be extended by Apply Discount, but only when a discount code is entered, so the extension only happens some of the time.

The two relationships are easy to confuse because both are drawn as dashed arrows. The direction of the arrow, and the everyday meaning of "always" versus "sometimes," is what distinguishes them, as shown in Figure 2.3.

Include vs Extend Relationships



Figure 2.3: Include (always required) versus extend (conditionally added)

Building a Use Case Diagram

The steps typically followed when building a use case diagram are:

1. Identify the actors: the people or external systems that will use the system
2. Identify the use cases: the tasks each actor needs to perform with the system
3. Draw an association line from each actor to every use case they are involved in
4. Identify any include or extend relationships between use cases, and draw them as dashed arrows labelled accordingly

Worked Example

Scenario: A school wants a grading system. A teacher can record and update grades. The administrator can generate report cards, but these are distributed by a teacher. A teacher, an administrator, and a student can all view grades, and viewing grades first requires logging in.

Applying the four steps above: the actors identified are Teacher, Administrator, and Student (step 1). The main use cases are Record Grades, Update Grades, Distribute Report Cards, Generate Report Cards, and View Grades (step 2). Each actor is then connected by an association line to the use cases they perform (step 3). Finally, Record Grades and Update Grades both include a Save/Load Grades use case, and View Grades includes a Login use case, since a user must be logged in before grades can be viewed (step 4). The resulting diagram is shown in Figure 2.4.

Use Case Diagram: School Grading System

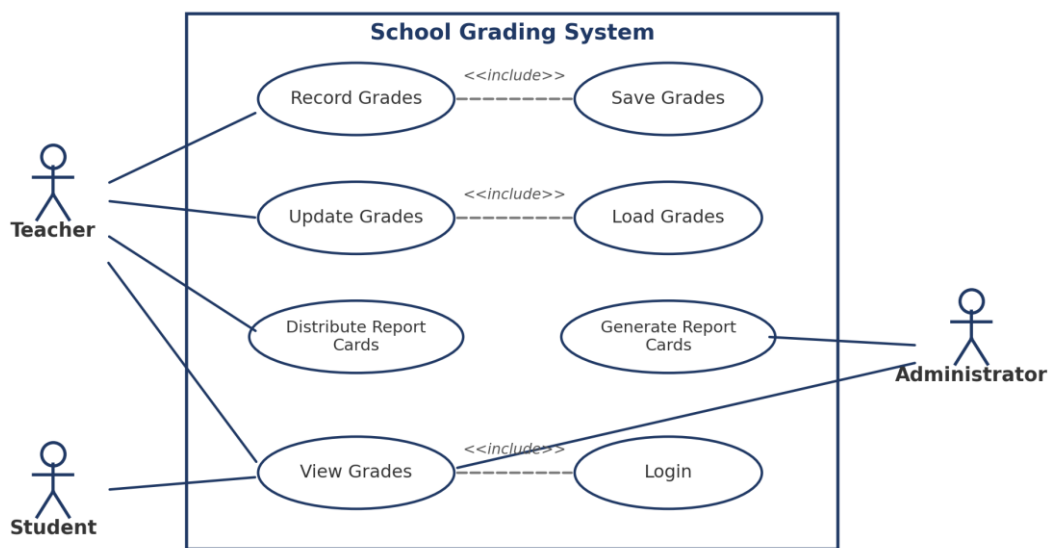


Figure 2.4: Use case diagram for the school grading system

Summary

This unit covered two central UML diagrams. The class diagram shows the static structure of a system through classes, their attributes, operations, and relationships (association, aggregation, composition, and inheritance). The use case diagram shows how actors interact with the system through use cases, connected by include and extend relationships. Both diagrams are usually developed together, since the classes identified must be consistent with the use cases already modelled.

Self-Assessment Questions

1. Explain the difference between aggregation and composition, with an example of each.
2. Explain the difference between an include relationship and an extend relationship in a use case diagram.
3. Draw a class diagram for a simple library system with the classes Book, Member, and Loan, showing appropriate relationships and multiplicity.

Unit Activities

1. Redraw the class diagram in Figure 2.2 from memory, then check it against the original (20 minutes)
2. Redraw the use case diagram in Figure 2.4 from memory, then check it against the original (20 minutes)

Tutor-Marked Assignment

1. You are to develop a hotel booking system through which a guest can search for rooms, make a booking, and make a payment. A hotel manager can add and remove rooms from the system.
 - a. Following the four steps in section 2.2.1, sketch a use case diagram for this scenario, showing all actors and use cases, and any include or extend relationships you identify.
2. Using the same hotel booking scenario:
 - a. Following the five steps in section 2.1.2, draw a class diagram for the scenario, showing at least four classes with their attributes, operations, and relationships.

References/Further Readings

O'Docherty, M. (2005). Object-Oriented Analysis and Design: Understanding System Development with UML 2.0. John Wiley & Sons.

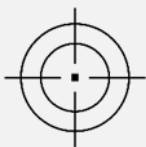
Dennis, A., Wixom, B. H., & Roth, R. M. (2012). Systems Analysis and Design (5th ed.). John Wiley & Sons.

Unit 3 — Behavioural UML Diagrams

INTRODUCTION

Track a food delivery order on any delivery app, and you will watch it move through a series of states: Placed, Preparing, Out for Delivery, Delivered. That progression is exactly what a state chart diagram models. Meanwhile, the decision the app makes behind the scenes, such as whether a rider is available nearby, is exactly the kind of branching logic an activity diagram captures. This unit works through both diagrams, plus the remaining UML diagrams needed to describe a complete system.

This unit completes the coverage of UML by looking at the remaining behavioural and structural diagrams: the sequence diagram, state chart diagram, activity diagram, component diagram, deployment diagram, and collaboration diagram. Each diagram gives a different view of how a system behaves or is physically arranged, and a full design typically uses several of them together, since no single diagram tells the whole story.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Describe the purpose of a sequence diagram and construct a simple example
- Describe the purpose of a state chart diagram and an activity diagram
- Describe the purpose of a component diagram and a deployment diagram
- Describe the purpose of a collaboration (communication) diagram.

3.3.1 Sequence Diagram

A sequence diagram describes the flow of messages, events, and actions between objects over time. It shows how objects interact with one another in a particular scenario of a use case and is typically used during analysis and design to document and understand the logical order in which messages are exchanged, which is especially useful for scenarios that involve several systems talking to each other, such as a payment being authorised.

In a sequence diagram, each object involved is shown at the top of a vertical dashed line, called a lifeline. Messages are drawn as horizontal arrows between lifelines, ordered from top to bottom to show the sequence in which they occur. A solid arrow usually represents a call or request, while a dashed arrow represents a return or response, so a quick glance at the arrow style tells you whether something is being asked for or being handed back.

Building a Sequence Diagram

1. Identify the objects involved in the scenario, and place each one at the top of its own lifeline
2. List the messages that must be exchanged, in the order they occur
3. Draw each message as a horizontal arrow at the appropriate height, using a solid arrow for a request and a dashed arrow for a response
4. Number the messages if the diagram needs to make the sequence explicit, or rely on the vertical position alone if the order is already clear.

Figure 3.1 applies these four steps to an ATM cash withdrawal: the objects are Customer, ATM System, and Bank Server; the messages run from inserting a card through to debiting the account; and each message is drawn as an arrow at the point in time it occurs, reading from top to bottom.

Sample Sequence Diagram: ATM Cash Withdrawal

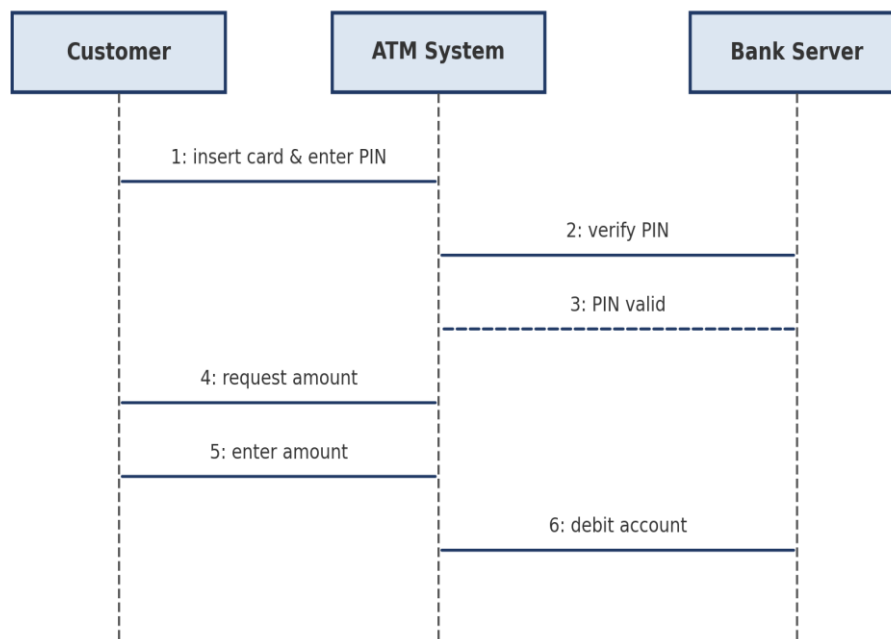


Figure 3.1: A sequence diagram for an ATM cash withdrawal

When a sequence diagram becomes too large, or needs to refer to another diagram, this can be shown using an unfinished arrow with a comment, or a labelled reference frame that names the other diagram, rather than redrawing the same interaction twice.

3.3.2 State Chart Diagram

A state chart diagram (or state machine diagram) shows the different states an object can be in during its lifetime, and the events that cause it to move from one state to another. It is useful for objects whose behaviour depends heavily on their current state, such as an Order object, which behaves quite differently once it has been Shipped compared to when it was only just Placed. Each state is drawn as a rounded rectangle, and transitions between states are drawn as arrows labelled with the event that triggers the change, starting from a solid black circle that marks the initial state.

Figure 3.2 shows the states of an Order object: it begins at Placed, moves to Confirmed once payment succeeds, then to Shipped, and finally to Delivered; it can also move to a Cancelled state if the customer cancels or the payment fails, showing that not every object follows a single straight-line path through its states.

State Chart Diagram: Order Object

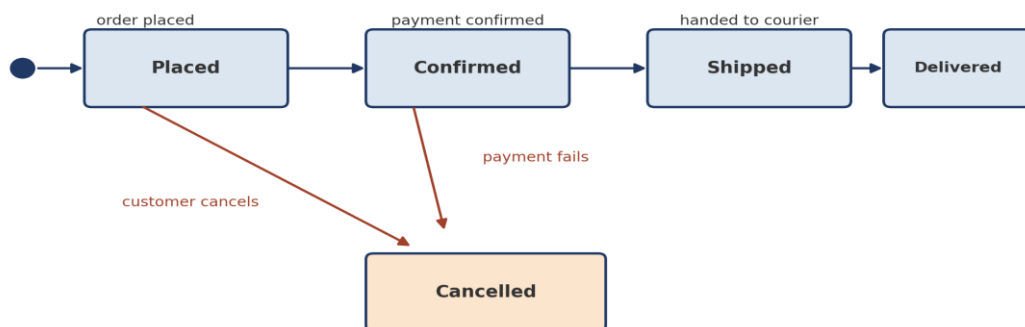


Figure 3.2: State chart diagram for an Order object

3.3.3 Activity Diagram

An activity diagram shows the flow of control, and optionally data, through the steps in a process. It is similar in spirit to a flowchart, and is useful for modelling workflows, business processes, and the logic within a single use case or operation. An activity diagram is made up of activity nodes, representing the execution of a step, connected by control flows and data flows. It can also show decision points, drawn as diamonds, where the flow branches depending on a condition, and parallel paths, where two or more activities happen at the same time.

Figure 3.3 models a room booking process: after searching and selecting a room, the flow reaches a decision point asking whether the room is available; if yes, the process continues to payment and confirmation; if no, the process notifies the user and ends without a booking.

Activity Diagram: Room Booking Process

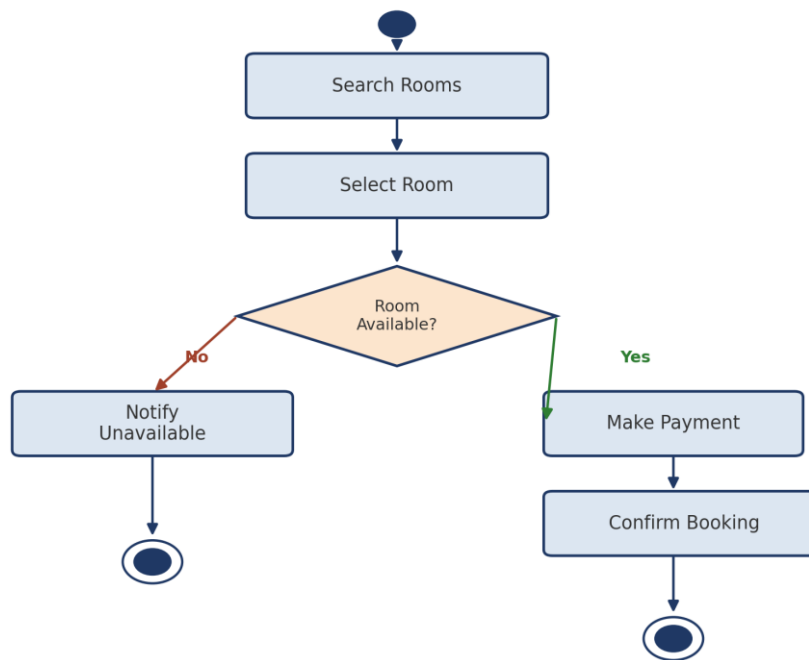


Figure 3.3: Activity diagram for a room booking process

3.3.4 Component Diagram

A component diagram depicts the physical architecture of the software: the software components that make up the system, and the relationships (usually dependencies) between them. A component is a replaceable, self-contained part of the system that provides a defined set of services through one or more interfaces, such as a payment processing component or a reporting component, and can often be replaced with an alternative implementation as long as it honours the same interface.

Figure 3.4 shows three components of a booking system: the Booking Component depends on the Payment Component to take payment, which in turn depends on the Notification Component to alert the customer once payment succeeds.

Component Diagram: Booking System



The small connector icons represent the interfaces each component provides to, or requires from, other components.

Figure 3.4: Component diagram for a booking system

3.3.5 Deployment Diagram

A deployment diagram describes the physical deployment architecture of the hardware and software that make up the system. It shows the processors, devices, and software components, and how they are connected, indicating where each part of the system runs. A deployment diagram is the more detailed, implementation-level view of a system's overall architecture: the client devices, application servers, and database servers, and how they are physically connected.

Figure 3.5 shows a typical deployment for a web application: a client device running the browser or mobile app, connected over HTTPS to an application server that runs the business logic, which is in turn connected over SQL to a database server that hosts the DBMS.

Deployment Diagram: Web Application

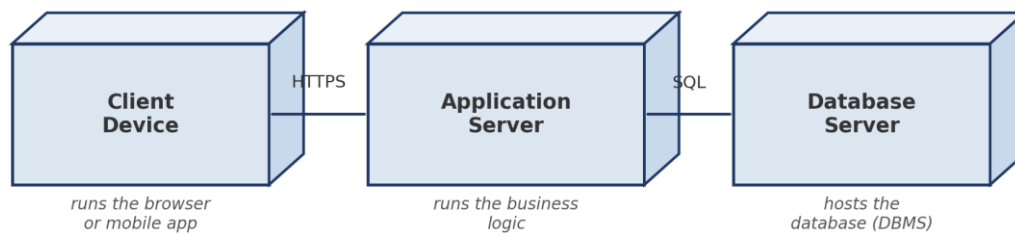


Figure 3.5: Deployment diagram for a web application

3.3.6 Collaboration (Communication) Diagram

A collaboration diagram, also called a communication diagram, shows the same information as a sequence diagram, objects and the messages exchanged between them, but organises it spatially around the objects rather than chronologically down the page. Messages are numbered to show their order. Collaboration diagrams are useful when the focus is on how objects are structurally connected, while sequence diagrams are more useful when the focus is on the exact timing and order of messages.

Figure 3.6 shows the same ATM cash withdrawal scenario as Figure 3.1 but arranged around the three participating objects instead of down a timeline, with each message numbered to preserve the order.

Collaboration Diagram: ATM Cash Withdrawal

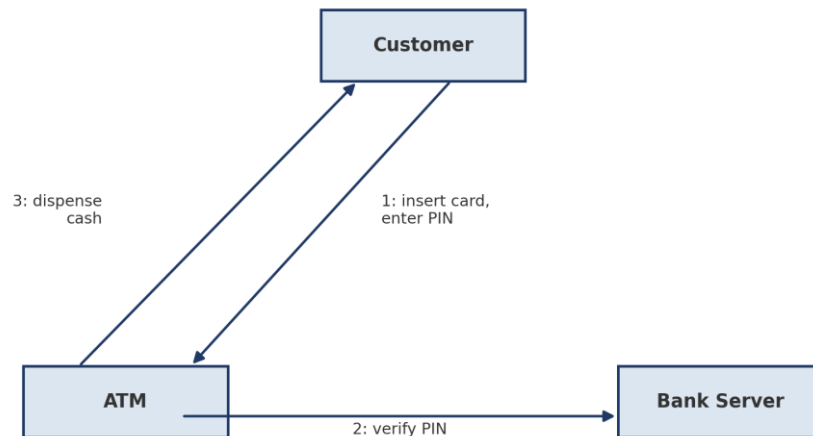


Figure 3.6: Collaboration diagram for an ATM cash withdrawal

Summary

This unit completed the survey of UML diagrams. The sequence diagram shows the chronological order of messages between objects for a specific scenario. The state chart diagram shows the states an object passes through during its life. The activity diagram shows the flow of control through a process, like a flowchart. The component diagram and deployment diagram both describe physical structure, the software components and the hardware they run on. The collaboration diagram shows the same interactions as a sequence diagram but organised around the objects rather than around time. Together with the class and use case diagrams from Unit 2, these diagrams give a complete object-oriented picture of a system, ready to be built using tools such as Microsoft Visio.

Self-Assessment Questions

1. Explain the difference between a sequence diagram and a collaboration diagram.
2. Explain, with an example, when a state chart diagram would be more useful than an activity diagram.

3. Distinguish between a component diagram and a deployment diagram.
4. Draw a sequence diagram for a student logging into a course portal and viewing their results.

Unit Activities

1. Draw a state chart diagram for a Library Book object, showing states such as Available, On Loan, Reserved, and Lost (25 minutes)
2. Sketch a simple deployment diagram for a web application, showing a client device, a web server, and a database server (20 minutes)

Tutor-Marked Assignment

1. Using the hotel booking scenario from Module Three, Unit 2 (a guest searches for rooms, makes a booking, and makes a payment):
 - a. Following the four steps in section 3.1.1, draw a sequence diagram showing the interaction between the Guest, the Booking System, and the Payment Gateway for a successful booking.
 - b. Draw an activity diagram showing the steps and decision points involved in making a booking, from search to payment confirmation.
2. Still using the hotel booking scenario:
 - a. Sketch a deployment diagram showing how you would physically deploy this system across a client device, an application server, and a database server.
 - b. Draw a state chart diagram for a Booking object, showing at least four states from creation to completion, including a cancelled path.

References/Further Readings

- O'Docherty, M. (2005). Object-Oriented Analysis and Design: Understanding System Development with UML 2.0. John Wiley & Sons.
- Bennett, S., McRobb, S., & Farmer, R. (2002). Object-Oriented Systems Analysis and Design Using UML (2nd ed.). McGraw-Hill.

